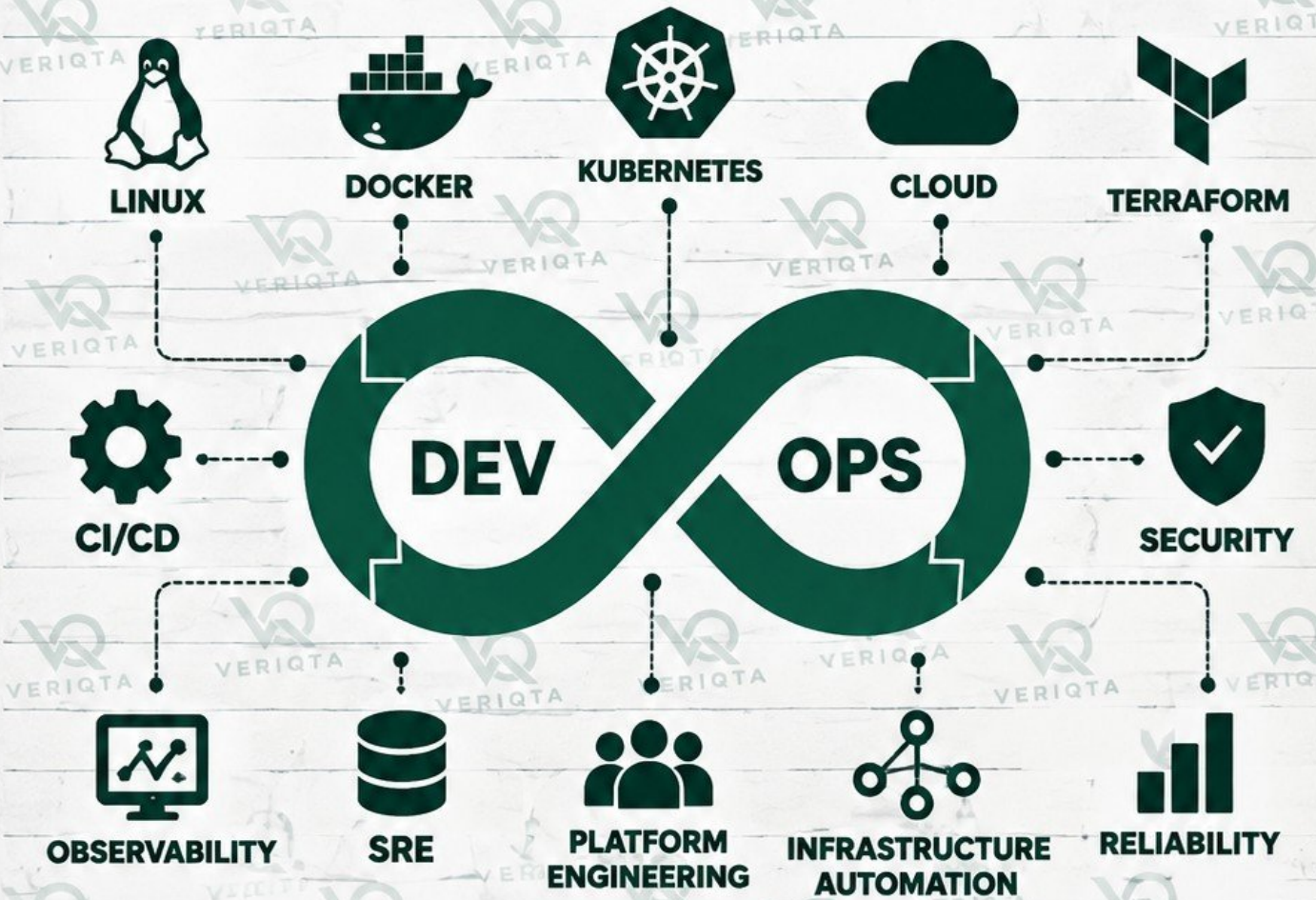
VERIQTA

DevOps Roadmap

for Mid-Level to Senior Engineers



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

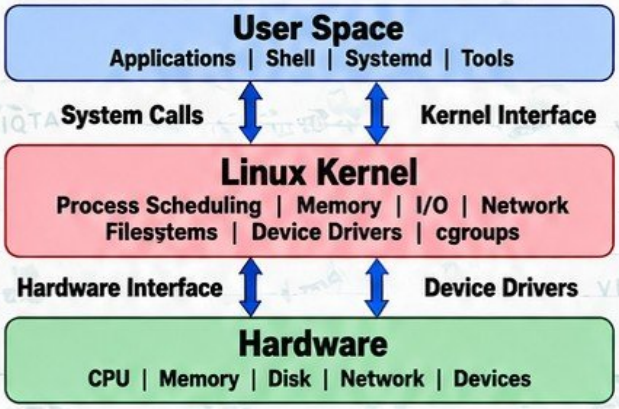
Page 1

VERIQTA

Linux Internals and Production Troubleshooting

Process scheduling, CPU contention, memory pressure, and OOM behavior.
File descriptors, systemd, journald, cgroups, and kernel limits.
Diagnose production performance degradation using evidence.

1. Linux Architecture Overview



2. Process Scheduling and CPU Contention

- Linux uses the Completely Fair Scheduler (CFS).
- Gives CPU time based on process priority and load.
- High CPU usage can cause run queue to grow, increasing request latency.
- CPU contention happens when more tasks are runnable than available CPU cores.

Check CPU and run queue

```
top
htop
uptime
cat /proc/loadavg
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu
```

High load average (> number of CPU cores) indicates CPU contention.

3. Memory Presure and OOM Behavior

- Linux uses virtual memory with page cache, buffers and swap.
- When memory is exhausted, the kernel may trigger the OOM (Out Of Memory) killer.
- OOM killer terminates processes to protect the system.

Check memory usage

```
free -h
cat /proc/meminfo
vmstat 1 5
dmesg | grep -i oom
journalctl -k | grep -i "out of memory"
```

Look for OOM kill evidence

Terminated process will show in dmesg or journald with "killed process" message.

4. File Descriptors and Kernel Limits

- Each process has a limit on open file descriptors (FDs).
- Exhausted FDs can cause errors like "Too many open files".
- Important for high-traffic services (web servers, databases).

Check file descriptor usage

```
ulimit -n
cat /proc/sys/fs/file-max
lsof -p <PID> | wc -l
ls /proc/<PID>/fd | wc -l
```

Error symptoms

- Too many open files
- Failed connections
- Service becomes unresponsive

5. systemd and Service Management

- systemd manages services, processes, and system resources.
- Useful for checking service status, logs and resource usage.

Common systemd commands

```
systemctl status <service>
systemctl restart <service>
systemctl enable <service>
journalctl -u <service> -n 100
systemctl list-units --type=service
```

Check resource usage

```
systemd-cgtop
systemctl show <service>
--property=TasksCurrent
--property=MemoryCurrent
```

6. journald and Log Analysis

- journald collects logs for kernel, system and services.
- Useful for troubleshooting failures and OOM events.

View and filter logs

```
journalctl -xe
journalctl -u <service> -n 100
journalctl -k | grep -i error
journalctl --since "1 hour ago"
journalctl -f
```

Common log sources

- Kernel logs
- Service logs
- OOM killer logs
- System startup logs
- Reboot and crash logs

7. cgroups and Resource Control

- cgroups limit and isolate CPU, memory, I/O and other resources.
- Used by systemd, Docker and Kubernetes.

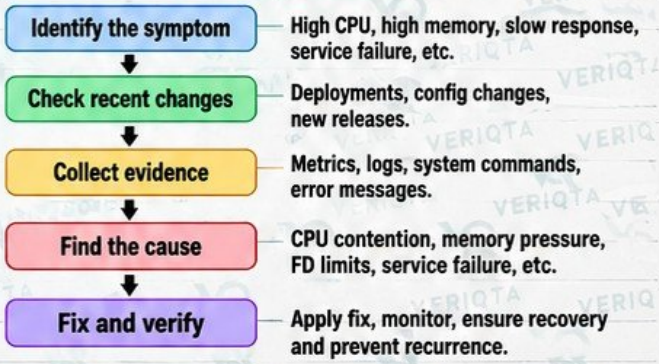
Inspect cgroups

```
ls /sys/fs/cgroup/
cat /sys/fs/cgroup/memory/memory.limit_in_bytes
cat /sys/fs/cgroup/memory/memory.usage_in_bytes
systemd-cgtop
```

Why it matters

- Prevent noisy neighbours
- Protect critical services
- Enforce resource limits

8. Troubleshooting Flow



9. Key Takeaways for Production

- Use evidence, not assumptions.
- Monitor CPU, memory, disk, network and FD usage.
- Understand systemd, journald and cgroups.
- Investigate before restarting services.
- Fix the root cause and add monitoring to prevent recurrence.

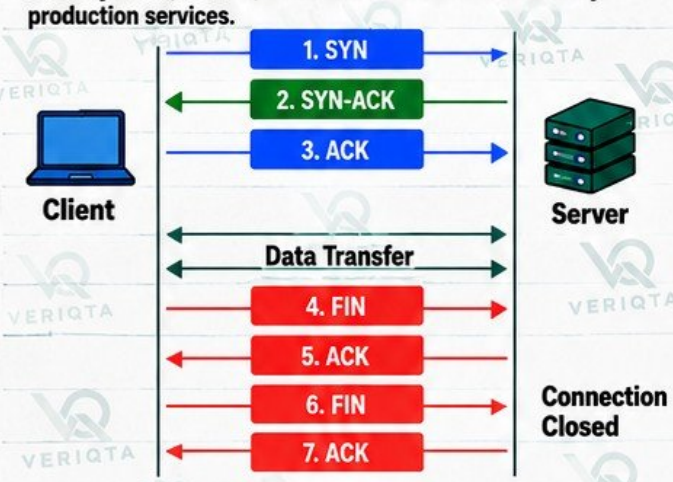
VERIQTA

Advanced Networking and Distributed Connectivity

TCP connection lifecycle, DNS, TLS, routing, NAT, and load balancing.
Connection pooling, timeouts, retries, packet loss, and MTU failures.
Investigate intermittent connectivity and network latency.

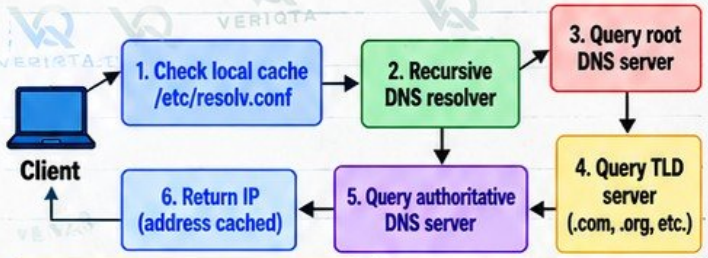
1. TCP Connection Lifecycle

- TCP creates a reliable, ordered, bidirectional connection.
- Used by HTTP, HTTPS, database connections and many production services.



2. DNS Resolution Flow

- DNS converts a domain name to an IP address.
- Production issues: slow DNS, wrong record, TTL, or provider outage.



Useful DNS commands

```
nslookup example.com
dig example.com
dig example.com +trace
dig example.com +short
cat /etc/resolv.conf
```

Common DNS issues

- High DNS latency
- DNS timeouts
- Wrong IP returned
- Expired records
- Local DNS misconfiguration
- Provider outage

3. TLS Handshake (HTTPS)

- TLS encrypts data and verifies server identity using certificates.
- Used in production for secure communication (HTTPS, APIs, DBs).
- Failed handshakes can cause connection errors and high latency.



Check TLS details

```
openssl s_client -connect example.com:443
curl -v https://example.com
openssl x509 -in cert.pem -text -noout
```

Common TLS issues

- Certificate expired
- Wrong certificate (hostname mismatch)
- Unsupported TLS version
- Wrong cipher suite
- Intermediate cert missing

4. Routing, NAT and Load Balancing

- Routing moves packets between networks.
- NAT translates private IPs to public IPs.
- Load balancers distribute traffic across healthy backends.



Useful commands

```
ip route show
traceroute example.com
curl ifconfig.me
ss -tuln
ip addr show
```

Common issues

- Wrong routing table
- NAT gateway failure
- Load balancer health check failure
- Asymmetric routing
- Security group / firewall block

5. Connection Pooling, Timeouts and Retries

- Connection pools reuse existing connections to reduce latency.
- Timeouts prevent hanging requests.
- Retries help with transient failures but can create new problems if not configured correctly.

Common configuration (example)

```
Max pool size: 50
Connection timeout: 3s
Read timeout: 5s
Retry attempts: 3
Retry backoff: exponential (e.g. 1s, 2s, 4s)
```

Best practices

- Use sane timeouts
- Limit retry attempts
- Use exponential backoff
- Avoid retry storms
- Monitor connection pool usage
- Tune based on real traffic

6. Packet Loss, Latency and MTU Issues

- Packet loss and high latency cause slow or failed connections.
- MTU mismatch can break large packets (common with VPNs and cloud).

Check network health

```
ping example.com
mtr example.com
traceroute example.com
ss -ti
tcpdump -i eth0
ping -M do -s 1472 example.com
```

Common problems

- Packet loss
- High latency (congestion)
- MTU too large (fragmentation)
- VPN or tunnel MTU issues
- Firewall dropping packets
- Network device misconfiguration

7. Investigating Intermittent Connectivity

Identify the symptom	Check recent changes	Gather evidence	Test each layer	Find the root cause
• Timeouts	• Deployments	• Logs	• DNS	• Fix the issue
• High latency	• Config changes	• Metrics	• Network path	• Verify stability
• Connection resets	• Network updates	• Packet captures	• TLS handshake	• Add monitoring
• Intermittent failures	• DNS changes	• Traces	• Application	• Prevent recurrence

8. Key Takeaways for Production

- ✅ Understand how each layer (DNS, TCP, TLS, routing, LB) works.
- ✅ Use evidence to troubleshoot, not assumptions.
- ✅ Monitor latency, error rate and connection failures.
- ✅ Set appropriate timeouts and retry policies.
- ✅ Validate MTU and network configuration for VPN and cloud.
- ✅ Identify and fix the real root cause, not just the symptom.
- ✅ Add monitoring and alerts to prevent recurrence.

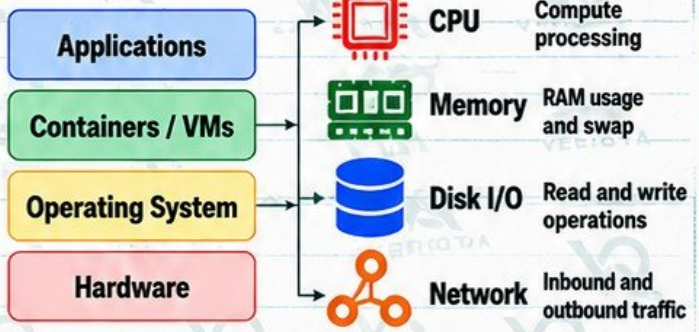
VERIQTA

Infrastructure Performance and Capacity Engineering

CPU throttling, memory saturation, disk I/O, and network bottlenecks.
Capacity forecasting, performance baselines, and resource contention.
Identify saturation before it causes customer-facing incidents.

1. System Performance Overview

- CPU, memory, disk and network all compete for resources.
- Saturation increases latency, errors and can cause downtime.
- Monitor, baseline and forecast to avoid customer impact.



2. CPU Throttling and Contention

- CPU throttling happens when a process exceeds its CPU limit.
- Causes higher latency and slower request processing.
- Common in containers, VMs and noisy neighbour environments.

Check CPU usage and throttling

```
top
htop
mpstat -P ALL 1
cat /sys/fs/cgroup/cpu/cpu.stat
kubectl top pods
kubectl describe pod <pod>
```

Signs of CPU contention

- High CPU usage
- High load average
- CPU throttling events
- Increased latency
- Run queue growing
- More context switching

Key metrics to monitor

- CPU usage (%)
- CPU throttling rate
- Load average
- Run queue length
- Context switches
- CPU steal (in VMs)
- Request latency

3. Memory Saturation and OOM

- Memory saturation causes swapping, high latency and OOM kills.
- Monitor both application and system memory usage.
- Understand memory behaviour in containers and VMs.

Check memory usage

```
free -h
vmstat 1 5
cat /proc/meminfo
dmesg | grep -i oom
kubectl top pods
kubectl describe pod <pod>
```

Signs of memory pressure

- High memory usage
- Swap usage increasing
- OOMKilled containers
- Slow response times
- Increased page faults
- Kernel OOM messages

4. Disk I/O Performance

- Slow disk I/O increases latency and affects application performance.
- Can be caused by high load, noisy neighbours or storage limits.
- Monitor IOPS, throughput and I/O wait.

Check disk I/O

```
iostat -x 1 5
iotop
df -h
lsblk
cat /proc/diskstats
dmesg | grep -i error
```

Signs of disk I/O issues

- High I/O wait (iowait)
- High disk latency
- High read/write queue
- Low disk throughput
- Filesystem nearing full
- Storage throttling (cloud)

5. Network Performance and Bottlenecks

- Network latency and packet loss cause slow or failed requests.
- Can be due to bandwidth limits, MTU issues, DNS, or network devices.
- Monitor traffic, errors, retransmits and latency between services.

Check network performance

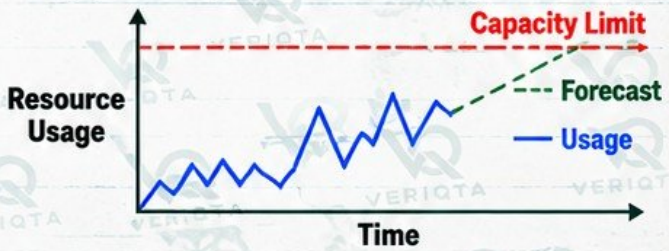
```
ping <host>
traceroute <host>
ss -tulnn
iftop
sar -n DEV 1 5
tcpdump -i eth0
```

Signs of network issues

- High latency
- Packet loss
- Retransmissions
- High error rate
- MTU mismatch
- Interface saturation

6. Capacity Forecasting and Baselines

- Use historical data to forecast future resource needs.
- Set performance baselines for normal and peak periods.
- Helps plan scaling, budget and avoid unexpected saturation.



7. Tools for Monitoring and Capacity Planning

- Prometheus Metrics
- Grafana Visualization
- Node Exporter System Metrics
- CloudWatch (Cloud)
- Datadog (Logs + Metrics)

8. Key Takeaways for Production

- ✅ Monitor CPU, memory, disk and network continuously.
- ✅ Understand baseline and peak usage.
- ✅ Set alerts before reaching saturation.
- ✅ Use evidence to investigate performance issues.
- ✅ Plan capacity based on real trends, not guesses.
- ✅ Scale proactively to prevent customer impact.

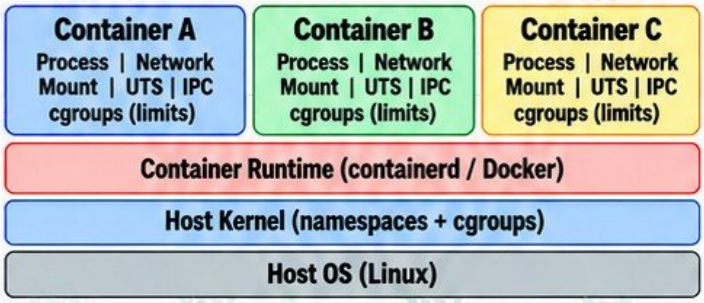
VERIQTA

Container Internals and Production Runtime Security

**Namespaces, cgroups, overlay filesystems, and container isolation.
Image layers, resource limits, runtime behavior, and security boundaries.
Debug container failures and reduce image supply-chain risks.**

1. How Containers Work Internally

- Containers share the host kernel but are isolated using namespaces and cgroups.
- Overlay filesystems provide a writable layer on top of read-only image layers.



2. Linux Namespaces (Isolation)

- Namespaces isolate processes, network, file system, users and more.
- Containers see their own view, but share the host kernel.

Namespace	Isolates	Example
PID	Process IDs	Each container has its own PID 1
NET	Network stack	Own interfaces, routing, ports
MNT	File system mounts	Separate root filesystem
IPC	Inter-process comms	Isolated shared memory, semaphores
UTS	Hostname and domain	Own hostname in container
USER	User and group IDs	Run as non-root (rootless)

3. Cgroups and Resource Control

- cgroups limit and track CPU, memory, I/O and other resources.
- Prevent noisy neighbours and improve stability.

Resource	cgroup Control	Example
CPU	cpu.max (v2)	Limit CPU usage
Memory	memory.max	Limit memory usage
I/O	io.max	Limit disk I/O
PIDs	pids.max	Limit number of processes

View cgroup settings

```
cat /sys/fs/cgroup/
ls /sys/fs/cgroup/
cat /proc/self/cgroup
docker inspect <id> |
grep -i cgroup
```

4. Overlay Filesystems and Image Layers

- Container images are made of read-only layers.
- A writable layer (container layer) sits on top using overlayfs.



View image layers

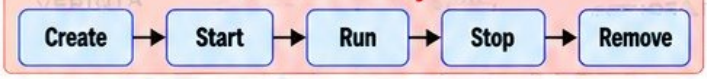
```
docker image inspect <image>
docker history <image>
docker inspect <id> |
grep -i "overlay"
```

5. Container Runtime and Lifecycle

- Container runtime (e.g. containerd) creates and manages containers.
- Understands images, isolation, resources and networking.



Container Lifecycle



6. Resource Limits and Runtime Behaviour

- Set CPU, memory and other limits to prevent resource exhaustion.
- Understand what happens when limits are exceeded.

Set resource limits

```
docker run -d \
  --cpus=1 \
  --memory=512m \
  --pids-limit=100 \
  --name app nginx
```

What happens on limit breach?

- CPU: throttling (not killed)
- Memory: OOMKilled (container stops)
- Disk I/O: slower performance
- PIDs: cannot create new processes
- Check container status and logs
  `docker ps -a | grep <name>`

7. Debugging Container Failures

- Use container logs, events and metrics to identify the root cause.
- Check resource usage, exit codes and kernel messages.

Useful debugging commands

```
docker ps -a
docker logs <id>
docker inspect <id>
docker top <id>
docker stats
docker exec -it <id> sh
dmesg | grep -i oom
```

Common failure causes

- OOMKilled (memory limit)
- CrashLoopBackOff (app crash)
- High CPU throttling
- Disk space full
- Too many open files
- Permission issues
- Network/DNS failures
- Application misconfiguration

8. Production Security and Supply-Chain Risk

- Use minimal base images and keep them updated.
- Scan images, enforce trusted sources and run as non-root.

Best practices

- ✅ Use minimal base images (e.g. distroless)
- ✅ Keep images updated
- ✅ Scan for vulnerabilities
- ✅ Run as non-root user
- ✅ Use read-only filesystem
- ✅ Drop unnecessary capabilities
- ✅ Sign and verify images (e.g. cosign)
- ✅ Use trusted registries and SBOMs

Check image security

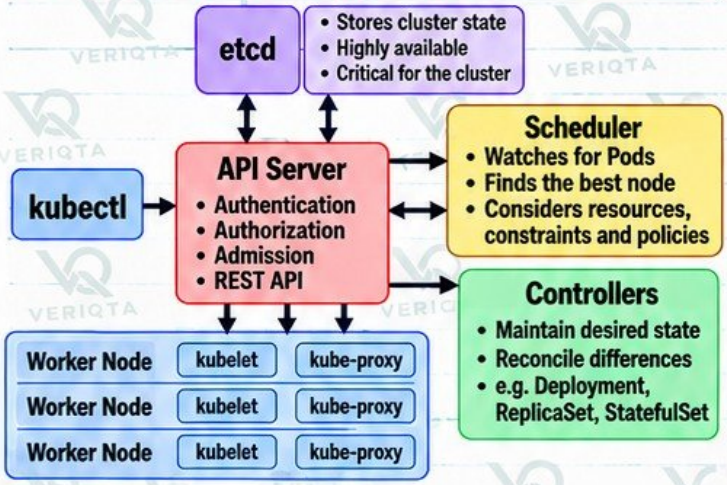
```
docker scan <image>
trivy image <image>
docker inspect <image> |
grep -i user
docker inspect <image> |
grep -i security
cosign verify <image>
```

VERIQTA

Kubernetes Control Plane and Reconciliation

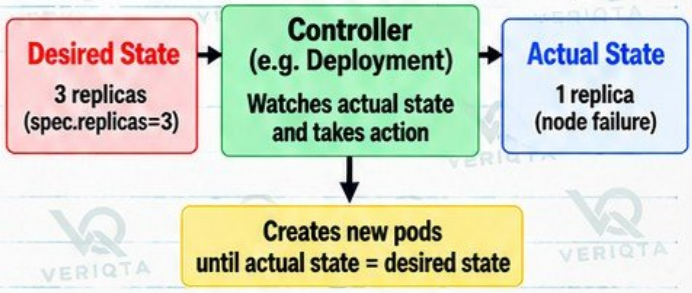
API server, etcd, scheduler, controllers, and desired-state reconciliation.
Scheduling decisions, admission, resource constraints, and control-plane pressure.
Diagnose why workloads remain Pending or fail to converge.

1. Control Plane Components



2. Desired-State Reconciliation

- You declare the desired state (e.g. 3 replicas).
- Controllers continuously watch and reconcile the actual state.
- If something changes (node failure, pod deleted), the controller creates new pods to match the desired state.



3. Scheduling Decisions

- Scheduler selects the best node based on available resources, constraints, policies and topology.
- Considers CPU, memory, node selectors, taints/tolerations, affinity/anti-affinity, and resource requests.

Factor	What it does	Example
Resource requests	Ensures enough CPU/memory	requests: cpu: 500m
Node selectors	Schedule on specific nodes	nodeSelector: env: prod
Taints and tolerations	Avoid or allow specific nodes	tolerations: key: gpu
Affinity / anti-affinity	Control pod placement	podAntiAffinity
Topology spread	Distribute across zones/nodes	topologySpreadConstraints

4. Admission Control and Validation

- Admission controllers validate and modify requests before objects are persisted in etcd.
- Used for security, policy enforcement and default settings.

Admission Type	Purpose	Examples
Mutating	Modify the request	Add labels, sidecars, default resources
Validating	Allow or reject the request	Enforce security policies, resource limits

Common controllers
- NamespaceLifecycle
- LimitRanger
- ResourceQuota
- PodSecurity (or PSA)
- ImagePolicyWebhook

Check admission logs

```
kubectl logs -n kube-system
-l component=kube-apiserver

kubectl get events -A | grep -i
"denied"
```

5. Common Reasons Pods Remain Pending

- Insufficient resources, taints, node selectors, PVC issues, or scheduler constraints.
- Use events and describe to find the exact reason.

Check why a pod is pending

```
kubectl get pods -A
kubectl describe pod <pod>
kubectl get events -A | tail -n 20
```

Common causes
- Insufficient CPU or memory
- Node taints not tolerated
- Node selector mismatch
- PVC not bound
- Quota or resource limit
- No available nodes
- Image pull issues

6. Control Plane Pressure and Symptoms

- High API request rate, many watches, large clusters or misbehaving controllers can put pressure on the control plane.
- Symptoms include slow kubectl, delayed scheduling and stale status.

Check control plane health

```
kubectl get componentstatuses
kubectl top nodes -n kube-system
kubectl get --raw "/metrics" | grep apiserver
kubectl get events -A | grep -i apiserver
```

Signs of control plane pressure
- High API latency
- Many pending requests
- Slow scheduler decisions
- etcd high disk or CPU
- Frequent timeouts
- Delayed reconciliations

7. Key Troubleshooting Commands

Purpose	Command
Cluster info	kubectl cluster-info
Check nodes	kubectl get nodes -o wide
Check control plane pods	kubectl get pods -n kube-system
Describe pod (events)	kubectl describe pod <pod>
Check scheduler logs	kubectl logs -n kube-system -l component=kube-scheduler
Check controller logs	kubectl logs -n kube-system -l component=kube-controller-manager
Check etcd health	kubectl get --raw "/healthz/etcd"

8. Key Takeaways for Production

- ✅ Understand how the control plane components work together.
- ✅ Monitor API server, scheduler, controller and etcd health.
- ✅ Set appropriate resource requests and limits.
- ✅ Use taints, node selectors and topology spread for reliable scheduling.
- ✅ Investigate Pending pods using events and describe.
- ✅ Watch for control plane pressure in large clusters.
- ✅ Use policies and admission controllers for security and governance.
- ✅ Ensure etcd is highly available and backed up.
- ✅ Automate monitoring and alerts to detect issues early.

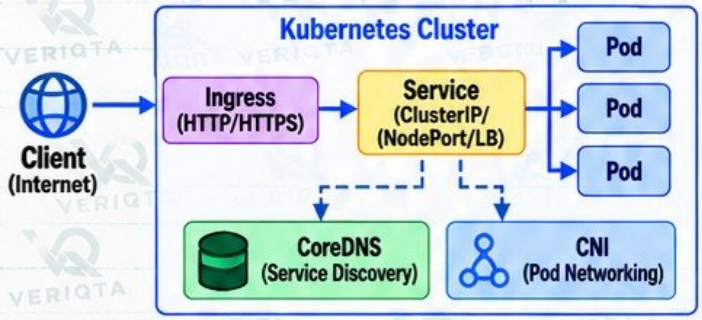
VERIQTA

Kubernetes Networking and Service Reliability

CNI, Services, EndpointSlices, CoreDNS, Ingress, and traffic routing.
NetworkPolicy, service discovery, readiness, and connection failures.
Investigate cross-service communication failures and traffic blackholes.

1. Kubernetes Networking Architecture

- Pods get IPs from the CNI and communicate across nodes.
- Services provide stable networking and load balancing.
- CoreDNS handles service discovery (DNS).
- Ingress exposes services outside the cluster.



2. Kubernetes Service Types and Traffic Flow

- Services provide a stable virtual IP and load balancing.
- EndpointSlices track healthy pod endpoints.
- kube-proxy or CNI handles the actual traffic routing.

Service Type	Use Case	How it works	Example
ClusterIP	Internal access within cluster	Virtual IP, load balances to pods	api:80
NodePort	Access from outside (testing)	Opens port on each node	30080 → :80
LoadBalancer	External access in cloud	Creates a cloud LB	AWS NLB/ALB
Headless	Direct pod DNS (no load balancing)	Returns pod IPs	clusterIP: None
ExternalName	Map to external DNS	CNAME to external service	db.example.com

3. CoreDNS and Service Discovery

- CoreDNS resolves service names to ClusterIP or pod IPs.
- Uses Kubernetes API to watch Services and EndpointSlices.
- DNS failures cause cross-service communication issues.

Test DNS resolution

```
kubectl run -it --rm dnsutils --image=busybox \\
  -- nslookup api.default.svc.cluster.local

dig api.default.svc.cluster.local
cat /etc/resolv.conf
```

Common DNS issues

- No DNS response
- Wrong IP address
- Slow DNS resolution
- CoreDNS pod failure
- Misconfigured upstream DNS

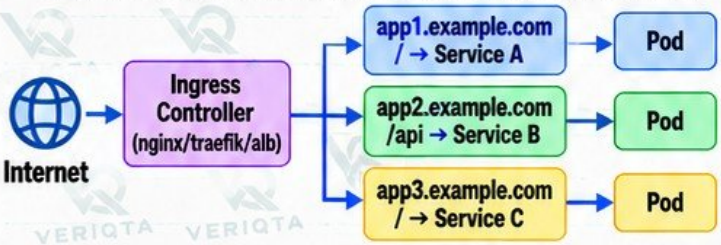
4. Container Network Interface (CNI)

- CNI assigns pod IPs and sets up networking between nodes.
- Popular CNIs: Calico, Cilium, Flannel, Weave.
- Handles pod-to-pod, network policies and routing.

CNI	Key Features	Use Case
Calico	NetworkPolicy, BGP	Production, security
Cilium	eBPF, high performance	Large clusters
Flannel	Simple overlay	Small clusters
Weave	Easy setup	Development/testing

5. Ingress and External Traffic Routing

- Ingress manages HTTP/HTTPS traffic to services.
- Uses an Ingress Controller (e.g. NGINX, Traefik, ALB).
- Supports host-based and path-based routing, TLS and rewrites.



6. NetworkPolicy for Traffic Control

- Controls which pods can communicate with each other.
- Default: all pods can communicate (unless policies are applied).
- Use least privilege and allow only required traffic.

Example: allow only frontend to backend

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
      - podSelector:
          matchLabels:
            app: frontend
    ports:
      - port: 8080
        protocol: TCP
```

Common NetworkPolicy issues

- Traffic blocked unexpectedly
- Missing egress rules
- Wrong pod labels/selectors
- DNS blocked (port 53)
- Overly restrictive rules

Test connectivity

```
kubectl exec -it <pod> --
  curl -v http://<service>:<port>
kubectl exec -it <pod> --
  nslookup <service>
kubectl exec -it <pod> --
  ping <pod-ip>
```

7. Investigating Connection Failures

- Check DNS, service, endpoints, network policies and pod health.
- Use logs, events and network tools to find the exact failure point.

Useful debugging commands

```
kubectl get pods -o wide
kubectl get svc
kubectl get endpointslices
kubectl get networkpolicy
kubectl describe pod <pod>
kubectl logs <pod>
kubectl exec -it <pod> -- curl -v <service>:<port>
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- traceroute <service>
```

Common causes

- DNS resolution failure
- Service with no endpoints
- NetworkPolicy blocking traffic
- Pod not ready
- Wrong port or protocol
- Ingress misconfiguration
- Node/CNI networking issue
- Firewall or cloud SG blocking
- MTU / packet fragmentation

8. Key Takeaways for Production

- ✅ Understand how CNI, Services, CoreDNS and Ingress work together.
- ✅ Use NetworkPolicy to enforce least privilege.
- ✅ Monitor CoreDNS, drop rate, and service endpoint health.
- ✅ Check DNS, endpoints, pod readiness and policies when debugging.
- ✅ Use meaningful labels and consistent naming.
- ✅ Validate connectivity between services before releases.
- ✅ Be aware of MTU, routing and cloud firewall rules.
- ✅ Monitor for traffic blackholes and unusual latency.
- ✅ Implement observability for network flows (metrics, logs, traces).

VERIQTA

Kubernetes Resource Management and Autoscaling

Requests, limits, QoS, eviction, HPA, VPA, and node autoscaling.
OOMKilled containers, CPU throttling, and scaling delays.
Balance workload performance, availability, and infrastructure cost.

1. Resource Requests and Limits

- Requests guarantee minimum resources for scheduling.
- Limits set the maximum resources a container can use.
- CPU is throttled when usage exceeds the limit.
- Memory limit breach can cause OOMKilled.

Setting	Purpose	Example
requests.cpu	Minimum CPU for scheduling	500m
limits.cpu	Maximum CPU (throttled if exceeded)	1
requests.memory	Minimum memory for scheduling	512Mi
limits.memory	Maximum memory (OOMKilled if exceeded)	1Gi

2. Kubernetes Quality of Service (QoS)

- QoS class decides eviction priority when the node is under pressure.
- Guaranteed: requests = limits.
- Burstable: requests < limits.
- BestEffort: no requests and no limits.

QoS Class	Configuration	Eviction Priority	Use Case
Guaranteed	requests = limits	Lowest (most protected)	Critical workloads
Burstable	requests < limits	Medium	General workloads
BestEffort	No requests No limits	Highest (evicted first)	Non-critical batch jobs

3. Pod Eviction and Node Pressure

- Nodes evict pods when resources are low (memory, disk, inode, PID).
- Kubelet monitors resource usage and triggers eviction.
- QoS class and resource usage determine which pods are evicted.

Check node pressure

```
kubectl describe node <node>
kubectl get events
--sort-by=.lastTimestamp
kubectl get pods -A -o wide
```

Common eviction reasons

- Memory pressure
- Disk pressure
- Inode pressure
- PID pressure
- Node not ready
- Image GC
- Kubelet eviction thresholds

4. OOMKilled Containers and CPU Throttling

- OOMKilled happens when a container exceeds its memory limit.
- CPU throttling happens when usage exceeds the CPU limit.
- Check logs, events and metrics to confirm the cause.

Check for OOMKilled

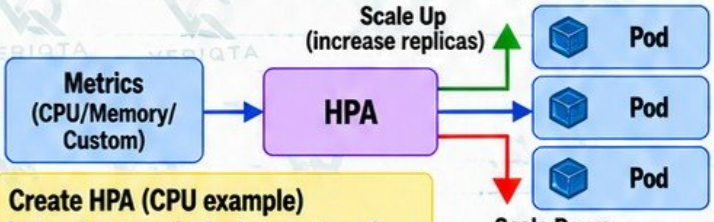
```
kubectl get pods -A
kubectl describe pod <pod>
kubectl logs <pod> --previous
dmesg | grep -i oom
```

Check CPU throttling

```
kubectl top pod <pod>
kubectl describe pod <pod>
cat /sys/fs/cgroup/cpu/
  cpu.stat
kubectl get events -A
```

5. Horizontal Pod Autoscaler (HPA)

- Automatically scales pods based on CPU, memory or custom metrics.
- Increases or decreases the number of pod replicas.
- Works with Metrics Server or external metrics (e.g. Prometheus).



Create HPA (CPU example)

```
kubectl autoscale deployment app \
  --cpu-percent=70 --min=2 --max=10
```

6. Vertical Pod Autoscaler (VPA)

- Automatically adjusts CPU and memory requests (and optionally limits) based on real usage.
- Helps right-size workloads and reduce resource waste.
- Can run in recommend, auto or initial mode.



Create VPA (recommendation mode)

```
kubectl apply -f vpa.yaml
```

VPA modes

- Off: only recommendations
- Initial: set at creation
- Auto: automatically update

7. Cluster Autoscaler (Node Autoscaling)

- Automatically adds or removes nodes based on pending pods and resource needs.
- Works with cloud providers (AWS, GCP, Azure).
- Helps handle spikes and reduces cost during low usage.



8. Best Practices and Troubleshooting

- ✅ Set realistic requests and limits based on real usage.
- ✅ Monitor CPU, memory, and node pressure continuously.
- ✅ Use HPA for variable traffic and VPA for right-sizing.
- ✅ Enable cluster autoscaler to handle node capacity.
- ✅ Investigate OOMKilled, throttling and eviction events.
- ✅ Use Prometheus/Grafana for resource monitoring.
- ✅ Avoid overprovisioning to reduce infrastructure cost.
- ✅ Test scaling behavior during load and failure scenarios.
- ✅ Review and adjust resource settings regularly.
- ✅ Balance performance, availability and cost.

VERIQTA

Kubernetes Production Incidents and Recovery

CrashLoopBackOff, node failures, failed rollouts, storage issues, and disruptions.
Events, metrics, logs, deployment history, and diagnostic commands.
Mitigate incidents, verify recovery, and prevent recurrence.

1. Common Kubernetes Production Incidents

- Applications can fail due to many different reasons.
- Understanding common incident types helps faster diagnosis.
- Always check events, logs, metrics and describe output.

CrashLoopBackOff	Node failure	Failed rollout	Storage issues	Disruptions
Container keeps restarting	Node NotReady or unreachable	Deployment not progressing	PVC/PV errors or volume mount failures	Network, DNS, API server or resource issues

2. CrashLoopBackOff Troubleshooting

- Happens when a container repeatedly starts and crashes.
- Caused by misconfiguration, missing dependencies, or app errors.
- Check logs, events and previous container logs.

Key commands

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp
```

Common causes

- App crash on startup
- Missing config/secrets
- Wrong environment vars
- Dependency not ready
- OOMKilled
- Liveness probe failure

3. Node Failures and NotReady Nodes

- Nodes can fail due to hardware issues, network problems or high resource usage.
- When a node is NotReady, pods are rescheduled to healthy nodes.
- Check node status, events, resource pressure and kubelet logs.

Check node status

```
kubectl get nodes -o wide
kubectl describe node <node>
kubectl get pods -o wide --all-namespaces
kubectl get events --field-selector
involvedObject.kind=Node
```

Common causes

- High CPU or memory pressure
- Disk pressure
- Network or DNS issues
- Kubelet not running
- Node NotReady
- Hardware failure

4. Failed Rollouts and Deployment Issues

- Rollouts can fail due to bad images, misconfigurations or failed probes.
- Use rollout history and events to identify the problem.
- Roll back if needed and fix the root cause.

Deployment troubleshooting

```
kubectl get deployments
kubectl describe deployment <name>
kubectl rollout status deployment
    <name>
kubectl rollout history deployment
    <name>
kubectl rollout undo deployment <name>
```

Common causes

- Bad container image
- Readiness/liveness probe failure
- Insufficient resources
- Configuration errors
- Image pull issues
- CrashLoopBackOff after update

5. Storage Issues (PVC/PV and Volume Mounts)

- Pods can fail to start if storage is not available or cannot be mounted.
- Check PVC, PV, storage class and events.
- Ensure the storage backend is healthy.

Check storage

```
kubectl get pvc
kubectl get pv
kubectl describe pvc <name>
kubectl describe pv <name>
kubectl get storageclass
kubectl get events
```

Common issues

- PVC stuck in Pending
- No matching storage class
- Insufficient storage
- Volume mount failures
- Permission issues
- Storage backend down
- ReadOnly file system

6. Events, Logs, Metrics and Diagnostic Flow

- Use a combination of events, logs, metrics and describe output.
- Follow a structured approach to find the root cause quickly.

1. Check Pod Status	`kubectl get pods -o wide`
2. Check Events	`kubectl get events --sort-by=.lastTimestamp`
3. Check Logs	`kubectl logs <pod>` `kubectl logs <pod> --previous`
4. Check Metrics	`kubectl top pods` `kubectl top nodes`
5. Investigate Further	`kubectl describe pod <pod>` `kubectl describe node <node>`

7. Mitigation and Recovery Steps

- Take quick action to restore service safely.
- Validate the fix and monitor for stability.

1. Identify the impact	Check which services and users are affected.
2. Mitigate	Rollback, scale, restart or move workload.
3. Fix the root cause	Update config, image, resources or storage.
4. Verify recovery	Check pod status, metrics and application health.
5. Monitor	Ensure stability and watch for recurrence.

8. Best Practices and Prevention

- ✅ Use readiness and liveness probes correctly.
- ✅ Set appropriate resource requests and limits.
- ✅ Monitor cluster, nodes, pods and storage continuously.
- ✅ Use alerts for critical events and resource pressure.
- ✅ Keep images, dependencies and configs tested.
- ✅ Use multiple replicas and spread across nodes and zones.
- ✅ Regularly review logs, events and metrics.
- ✅ Implement proper storage monitoring and backup.
- ✅ Run failure drills and document incident runbooks.
- ✅ Learn from incidents and improve configurations.

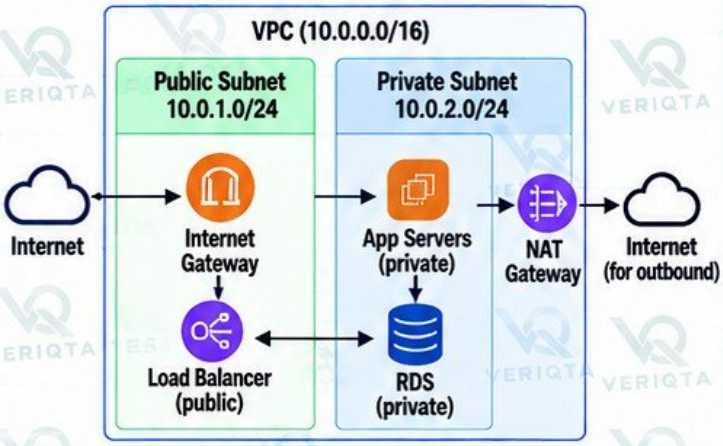
VERIQTA

Production Cloud Architecture and Failure Domains

VPC design, subnet isolation, routing, NAT, and private connectivity.
Multi-AZ architecture, regional dependencies, and high availability.
Evaluate resilience, cost, blast radius, and recovery trade-offs.

1. VPC Architecture and Network Design

- Design isolated and secure network architecture.
- Use public and private subnets, route tables and NAT.
- Control inbound and outbound traffic with security groups and NACLs.



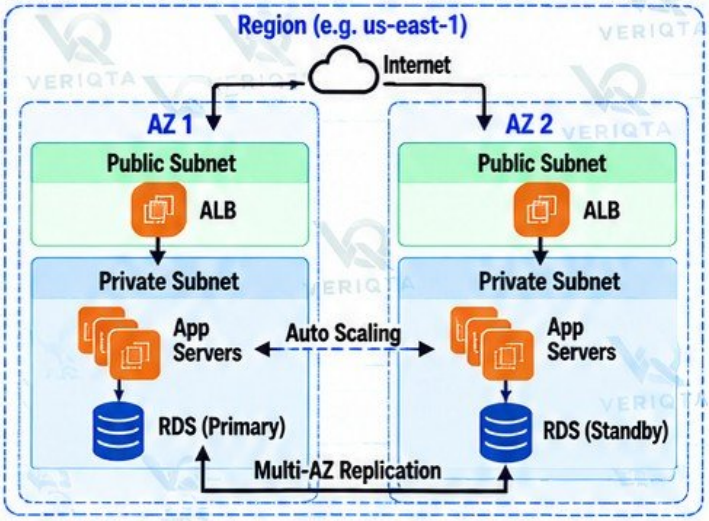
2. Subnets, Routing and Connectivity

- Public subnets: direct internet access (e.g. load balancers).
- Private subnets: no direct internet access.
- Route tables control traffic between subnets and external networks.

Component	Purpose	Example
VPC	Isolated network	10.0.0.0/16
Subnet	Network segment	10.0.1.0/24 (public)
Route Table	Controls routing	0.0.0.0/0 → IGW
Internet Gateway	Internet access	Attach to VPC
NAT Gateway	Outbound access for private subnets	In public subnet
Security Group	Instance-level firewall	Allow port 80, 443
Network ACL	Subnet-level firewall	Allow/deny CIDR

3. Multi-AZ Architecture (High Availability)

- Deploy resources across multiple Availability Zones (AZs).
- Use load balancers, Auto Scaling and managed databases.
- Ensure automatic failover and minimal downtime.



4. Failure Domains and Blast Radius

- Understand how failures in one component can affect others.
- Use isolation (AZs, subnets, accounts) to reduce blast radius.
- Design for fault tolerance and graceful degradation.

Failure Domain	Examples	Impact	Mitigation
Instance	EC2 failure	App unavailable	Auto Scaling
AZ	Power/network outage	One AZ down	Multi-AZ design
Region	Major regional outage	Entire region down	Multi-region architecture
Service	RDS, S3, etc.	Service disruption	Managed multi-AZ / backup
Network	Misconfigured routes, NACLs	Loss of connectivity	Testing, monitoring
Human error	Accidental deletion or bad config	Service outage	IAM, change control

5. Resilience, Cost and Recovery Trade-offs

- Balance high availability, performance, security and cost.
- Choose the right architecture based on business requirements.
- Plan for backups, disaster recovery and quick restoration.

Strategy	Benefit	Cost Impact	When to Use
Multi-AZ	High availability	Medium	Production systems
Multi-Region	Disaster recovery	High	Critical workloads
Auto Scaling	Handles traffic spikes	Low/Medium	Variable workloads
Managed Services	Less operational work	Medium	Most use cases
Backups	Point-in-time recovery	Low	All critical data
Infrastructure as Code	Repeatable and reliable	Low	All environments

6. Common Cloud Failure Scenarios

- Analyze real-world failure scenarios and their impact.
- Learn how to detect, respond and recover.

Scenario	Possible Cause	Impact
Loss of internet access	IGW or route misconfig	Users cannot access app
NAT Gateway failure	Single NAT in one AZ	Private resources lose outbound access
RDS primary failure	DB instance failure	Database unavailable
AZ outage	Power or network issue	One AZ down
Misconfigured security group	Wrong rules	Blocked traffic
Region outage	Major cloud outage	Entire region unavailable

7. Key Takeaways for Production

- ✅ Design secure and well-isolated VPC architecture.
- ✅ Use multi-AZ and multi-region for critical workloads.
- ✅ Understand and reduce blast radius.
- ✅ Monitor key components and set alerts.
- ✅ Plan for backup and disaster recovery.
- ✅ Regularly test failover and recovery procedures.
- ✅ Balance resilience, performance and cost.
- ✅ Document architecture and operational runbooks.

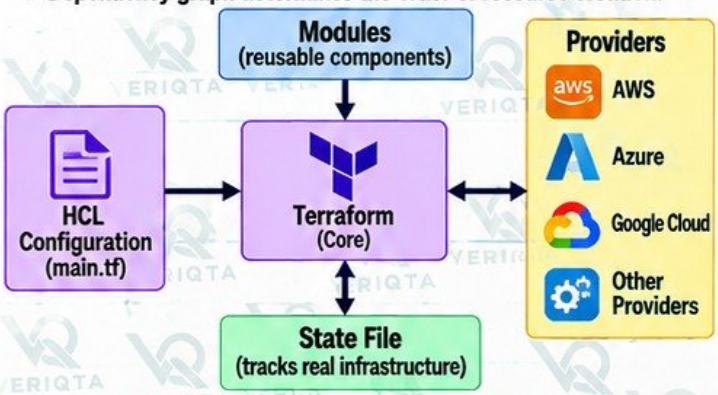
VERIQTA

Terraform Architecture and Infrastructure Governance

State management, locking, modules, providers, and dependency graphs.
Drift detection, destructive plans, lifecycle rules, and policy enforcement.
Design safe infrastructure changes with review and recovery controls.

1. Terraform Architecture and Components

- Terraform uses providers to manage infrastructure.
- Configuration is written in HCL and organized into modules.
- State file tracks real infrastructure and enables change planning.
- Dependency graph determines the order of resource creation.



2. State Management and Locking

- State file keeps a record of the real infrastructure.
- Use remote state with locking to prevent concurrent changes.
- Back up state, enable versioning, and restrict access.
- Never share state files publicly.

State Backend	Benefits	Example
Local (not recommended)	Simple, no locking	terraform.tfstate
S3 (AWS)	Remote, scalable, locking with DynamoDB	backend "s3" {}
Azure Storage	Remote, locking	backend "azurerm" {}
GCS (Google)	Remote, locking	backend "gcs" {}

Example: Remote state with locking (AWS S3 + DynamoDB)

```
terraform {
  backend "s3" {
    bucket         = "veriqta-terraform-state"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "veriqta-terraform-locks"
    encrypt        = true
  }
}
```

Best practices

- ✅ Use remote state
- ✅ Enable locking
- ✅ Use versioning
- ✅ Restrict access (IAM)
- ✅ Encrypt state file
- ✅ Back up regularly

3. Modules and Reusability

- Modules help you reuse and standardize infrastructure.
- Use input variables, outputs and version control.
- Choose between local modules, registry modules or custom modules.

Example: Using a module

```
module "vpc" {
  source  = "./modules/vpc"
  cidr    = "10.0.0.0/16"
  name    = "prod-vpc"
  tags = {
    Environment = "prod"
    Project     = "veriqta"
  }
}
```

Module structure

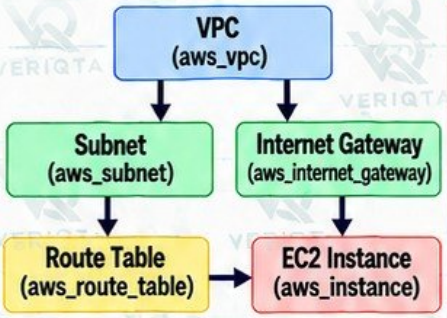
```
modules/
└ vpc/
    ├ main.tf
    ├ variables.tf
    └ outputs.tf
```

Benefits

- ✅ Reusable
- ✅ Consistent
- ✅ Easier maintenance
- ✅ Works across environments

4. Dependency Graph and Execution Order

- Terraform builds a dependency graph to determine resource order.
- Explicit and implicit dependencies ensure correct creation.
- Use depends_on only when necessary.



Key points

- Terraform creates resources in the correct order.
- Implicit dependencies via references.
- Use depends_on for explicit control.
- Avoid circular dependencies.

5. Drift Detection and Plan Safety

- Drift happens when real infrastructure is different from the state file.
- Always review terraform plan before apply.
- Use -refresh-only to check for drift without making changes.

Check for drift

```
terraform plan -refresh-only
terraform plan
terraform show
```

Prevent destructive changes

- Always review the plan.
- Use lifecycle rules.
- Use policy checks (e.g. Sentinel, OPA, Terraform Cloud).
- Set up required approvals.
- Use workspaces for isolation.
- Enable alerts for unexpected changes.

6. Lifecycle Rules and Policy Enforcement

- Use lifecycle rules to control resource behavior.
- Prevent accidental destruction and enforce policies.
- Integrate with Sentinel/OPA or Terraform Cloud for governance.

Example: Lifecycle rules

```
resource "aws_instance" "app" {
  ami           = "ami-12345678"
  instance_type = "t3.medium"
  lifecycle {
    prevent_destroy = true
    ignore_changes  = [user_data]
  }
}
```

Policy enforcement tools

- ✅ Terraform Cloud (Sentinel)
- ✅ OPA (Open Policy Agent)
- ✅ Checkov
- ✅ tfsec
- ✅ AWS Config / Azure Policy
- ✅ Cost and security policies

7. Safe Changes, Review and Recovery

1. Plan	Run terraform plan and review the changes.
2. Review	Have the plan reviewed (team or CI pipeline).
3. Apply	Apply changes with proper approvals.
4. Verify	Check resources and application health.
5. Recover	Use terraform state, backups or previous version to recover.

8. Key Takeaways for Production

- ✅ Use remote state with locking and versioning.
- ✅ Design reusable modules and keep code organized.
- ✅ Always review plans and detect drift regularly.
- ✅ Use lifecycle rules to prevent accidental destruction.
- ✅ Implement policy enforcement and approvals.
- ✅ Separate environments with workspaces and proper access control.
- ✅ Monitor infrastructure changes and set up alerts.
- ✅ Have a recovery plan and backups for critical resources.
- ✅ Balance automation, safety, cost and governance.

VERIQTA

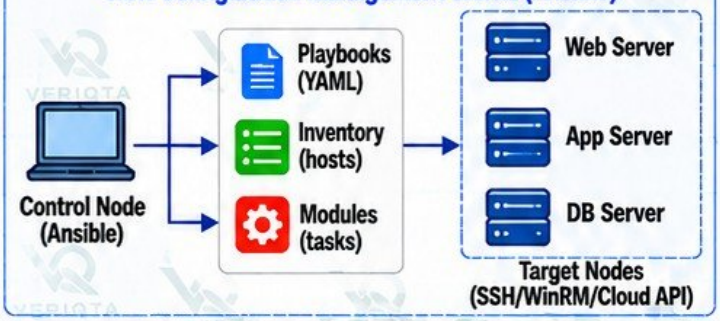
Infrastructure Automation and Configuration Management

Ansible, idempotency, immutable infrastructure, and configuration drift.
Secrets handling, inventory management, orchestration, and rollback.
Decide when configuration management or image replacement is appropriate.

1. What is Configuration Management?

- Automates the configuration of servers and infrastructure.
- Ensures consistent, repeatable and idempotent changes.
- Reduces manual work and configuration drift.
- Works well with immutable and hybrid infrastructure.

How Configuration Management Works (Ansible)



2. Ansible Architecture and Components

- Control node runs Ansible (no agent required).
- Uses SSH/WinRM/API to connect to target nodes.
- Playbooks define the desired state.
- Modules perform idempotent tasks.

Component	Purpose	Example
Control Node	Runs Ansible commands	Ansible on Linux
Inventory	List of target nodes	hosts.ini or dynamic
Playbook	Define tasks in YAML	site.yml
Module	Perform specific tasks	apt, yum, service, file
Role	Reusable set of tasks	nginx role
Collection	Community and official roles	ansible.posix
Vault	Encrypt sensitive data	ansible-vault
Facts	Gather system information	ansible_facts

3. Ansible Playbook Example

- Playbooks are written in YAML and define the desired state.
- Tasks are idempotent (safe to run multiple times).
- Use modules to install, configure and manage services.

site.yml

```
---
- name: Configure web servers
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Ensure Nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
```

Run the playbook

```
ansible-playbook site.yml
```

Check results

```
ansible all -m ping
ansible webservers -m service_facts
ansible webservers -m setup
```

4. Inventory Management

- Define target nodes statically or dynamically.
- Group servers by environment, role or region.
- Use variables for flexible configuration.

hosts.ini (static inventory)

```
[webservers]
web1 ansible_host=10.0.1.10
web2 ansible_host=10.0.1.11

[appservers]
app1 ansible_host=10.0.2.10
app2 ansible_host=10.0.2.11

[dbservers]
db1 ansible_host=10.0.3.10
db2 ansible_host=10.0.3.11
```

Inventory best practices

- ✅ Use groups (web, app, db)
- ✅ Use environment variables
- ✅ Use dynamic inventory (cloud, Kubernetes)
- ✅ Store in version control
- ✅ Keep sensitive data in Vault
- ✅ Use descriptive hostnames

5. Secrets Handling with Ansible Vault

- Encrypt sensitive data like passwords, API keys and certificates.
- Keep secrets in version control safely.
- Decrypt only when running playbooks.

Create an encrypted file

```
ansible-vault create secrets.yml
```

Edit an encrypted file

```
ansible-vault edit secrets.yml
```

Use in a playbook

```
vars_files:
  - secrets.yml
```

Example secrets.yml

```
db_password: "SuperSecure123"
api_key: "abcd1234"
aws_secret: "xyz9876"
```

Run with vault password

```
ansible-playbook site.yml
--ask-vault-pass
```

6. Idempotency and Configuration Drift

- Idempotency means running the same playbook does not cause unwanted changes.
- Detect and fix configuration drift automatically.
- Use Ansible to enforce the desired state regularly.

Example: idempotent task

```
- name: Ensure file exists
  ansible.builtin.file:
    path: /etc/app/config
    state: present
    owner: appuser
    mode: '0644'
```

Check for configuration drift

- ansible all -m setup
- ansible all -m file -a "path=/etc/app/config state=stat"
- Use --check (dry run)
  `ansible-playbook site.yml --check`

7. When to Use Configuration Management vs Image Replacement

- Use configuration management for existing servers, small changes and hybrid environments.
- Use image replacement (immutable infrastructure) for large changes, faster recovery and better consistency.

Approach	When to Use	Advantages	Limitations
Configuration Management	Small changes, existing servers, hybrid setups	Flexible, incremental changes, works anywhere	Can accumulate drift over time
Image Replacement (Immutable)	Large changes, cloud-native, containers	Consistent, fast recovery, no drift	Requires image pipeline and redeployment

8. Best Practices and Rollback

- ✅ Keep playbooks idempotent and well-structured.
- ✅ Use roles and collections for reusability.
- ✅ Store code and inventory in version control.
- ✅ Use Ansible Vault for secrets.
- ✅ Use dynamic inventory for cloud and Kubernetes.
- ✅ Test changes with --check before applying.
- ✅ Implement rollback plans for failed changes.
- ✅ Monitor configuration drift and enforce desired state.
- ✅ Use immutable infrastructure where possible.
- ✅ Document your automation and maintain change history.

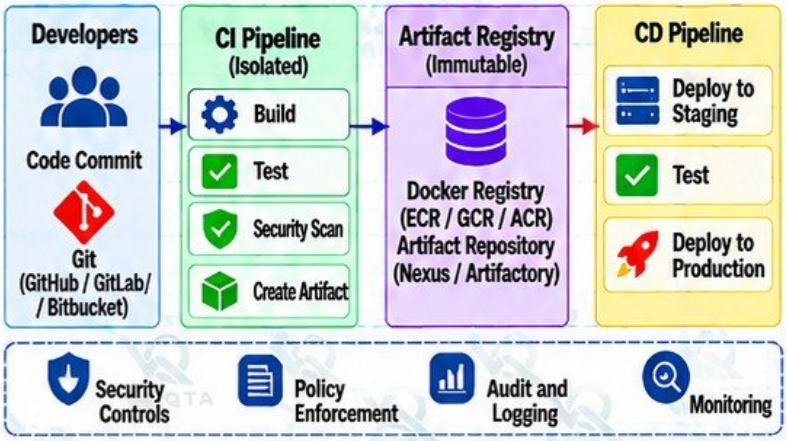
VERIQTA

Enterprise CI/CD Architecture and Supply-Chain Security

Pipeline isolation, artifact immutability, dependency verification, and SBOMs.
Secret protection, workload identity, signing, and provenance.
Build reproducible delivery pipelines with defensible trust boundaries.

1. Enterprise CI/CD Architecture (High Level)

- Isolated, secure and scalable pipeline architecture.
- Separation of build, test, and deployment environments.
- Use least privilege, network isolation and workload identity.
- Immutable artifacts and reproducible builds.



2. Supply-Chain Security Components

- Verify everything that goes into your pipeline.
- Use SBOMs, signing, and provenance to ensure integrity.
- Detect and block malicious or vulnerable dependencies.
- Maintain a trusted, auditable software supply chain.

Component	Purpose	Examples
Source Code	Version control and integrity	GitHub, GitLab, Bitbucket
Dependencies	Verify and scan packages	Snyk, Sonatype, Trivy
Build Environment	Isolated and reproducible builds	Ephemeral runners, containers
Artifact Registry	Immutable and signed artifacts	ECR, GCR, ACR, Artifactory
SBOM	List of all components	Syft, CycloneDX, SPDX
Signing	Cryptographic signing of artifacts	Cosign, Notary, Sigstore
Provenance	Track how artifacts are built	SLSA, in-toto, GitHub Attestations
Policy Enforcement	Block risky components	OPA, Conftest, Kyverno

3. Pipeline Isolation and Security Controls

- Use isolated runners and environments.
- Restrict network access and use least privilege permissions.
- Protect secrets and use workload identity instead of long-lived keys.
- Implement policy checks before deployment.

Isolated Runners
- Use ephemeral runners
- No access to production
- Restrict outbound access

Secret Protection
- Use secret managers
- Avoid storing secrets in code
- Use workload identity (OIDC)

Network Isolation
- Use private subnets
- Restrict egress traffic
- Use VPC endpoints

Policy Enforcement
- Scan code and images
- Enforce security policies
- Block deployments on failures

4. Artifact Immutability and Signing

- Build artifacts once and never modify them.
- Sign artifacts to verify authenticity.
- Use cryptographic signatures and provenance.
- Store artifacts in an immutable registry.

Example: Sign a container image with Cosign

```
# Login to registry
cosign login ghcr.io

# Sign the image
cosign sign ghcr.io/veriqta/app:1.0.0

# Verify the signature
cosign verify ghcr.io/veriqta/app:1.0.0
```

Benefits
- ✓ Prevents tampering
- ✓ Verifies authenticity
- ✓ Ensures immutability
- ✓ Supports audits
- ✓ Builds trust in delivery

5. Dependency Verification and SBOMs

- Scan and verify all dependencies (OS packages, libraries, containers).
- Generate SBOMs to track components.
- Check for vulnerabilities and blocked licenses.

Generate SBOM with Syft

```
syft packages \
 -o cyclonedx-json \
 app:1.0.0 > sbom.json
```

Scan image for vulnerabilities

```
trivy image app:1.0.0
```

View SBOM

```
cat sbom.json | jq .
```

What SBOMs Provide
- ✓ List of all components
- ✓ Versions and licenses
- ✓ Identify vulnerable packages
- ✓ Support compliance (e.g. NIST)
- ✓ Detect unauthorized components

6. Deployment with Policy and Provenance

- Use policy checks to validate images and configurations.
- Attach provenance to track how an artifact was built.
- Only allow signed and verified artifacts to be deployed.

Example: Verify image before deployment

```
# Verify signature
cosign verify ghcr.io/veriqta/app:1.0.0
# Check policy with OPA (Conftest)
conftest test deployment.yaml

# Deploy only if checks pass
kubectl apply -f deployment.yaml
```

Policy Checks Can Validate
- ✓ Image signatures
- ✓ Allowed registries
- ✓ Vulnerability thresholds
- ✓ Required labels
- ✓ Resource limits
- ✓ Approved configurations

7. When to Use Configuration vs Image Replacement

- Decide based on the type of change, risk and environment.
- Use configuration management for small, safe changes.
- Use immutable images for application and dependency changes.

Scenario	Best Approach	Reason
Update application code	Image replacement	Immutable, reproducible, lower risk
Change configuration file	Configuration management	Quick, small and safe change
Update system packages	Image replacement	Reproducible and consistent
Apply infrastructure changes	Configuration management	Manage declaratively (e.g. Ansible)
Emergency fix	Depends on risk	Use config for quick fix, image for complex changes

8. Best Practices and Key Takeaways

- ✓ Use isolated and ephemeral CI runners.
- ✓ Keep secrets in a dedicated secret manager.
- ✓ Verify dependencies and generate SBOMs.
- ✓ Sign artifacts and verify signatures.
- ✓ Enforce policies and block risky components.
- ✓ Use immutable artifacts and versioned deployments.
- ✓ Implement provenance and audit logging.
- ✓ Use workload identity instead of long-lived keys.
- ✓ Regularly scan for vulnerabilities and license issues.
- ✓ Design defensible trust boundaries across the pipeline.
- ✓ Test recovery and rollback procedures.
- ✓ Document and review your supply chain security.

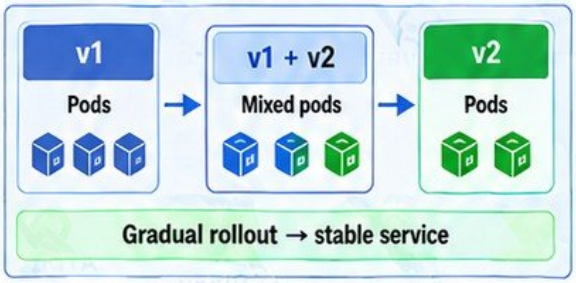
VERIQTA

Advanced Deployment Strategies and Release Safety

Rolling, blue-green, canary, and progressive delivery.
Feature flags, deployment health signals, automated rollback, and database compatibility.
Choose release strategies based on risk, observability, and recovery speed.

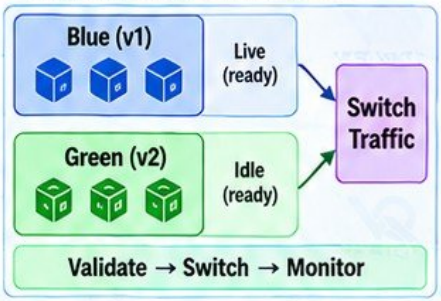
1. Rolling Deployment

- Gradual replacement of old pods with new ones.
- Minimizes downtime and maintains availability.
- Best for stateless applications.



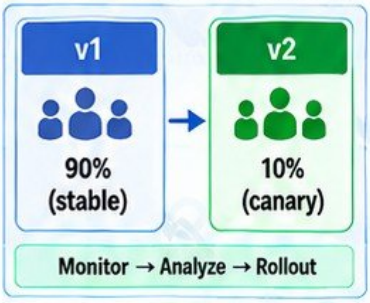
2. Blue-Green Deployment

- Run two identical environments.
- Switch traffic after validation.
- Easy rollback by switching back.



3. Canary Deployment

- Release to a small percentage first.
- Monitor metrics and user feedback.
- Gradually increase traffic if healthy.



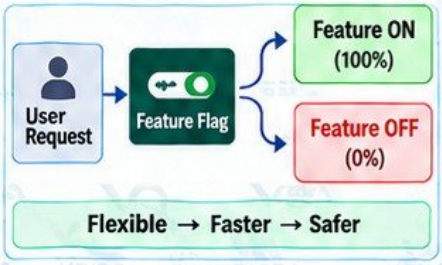
4. Progressive Delivery

- Gradual rollout with automation and analysis.
- Uses metrics, alerts and gates.
- Supports continuous feedback and improvement.



5. Feature Flags

- Control features without redeploying.
- Enable/disable via configuration.
- Useful for A/B testing and gradual rollout.



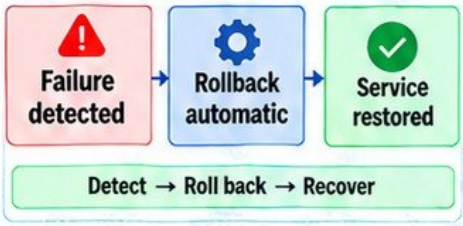
6. Deployment Health Signals

- Use readiness, liveness and custom checks.
- Monitor logs, metrics and error rates.
- Detect and stop bad releases early.



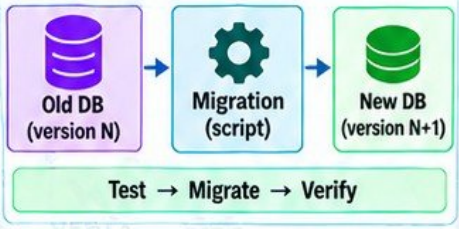
7. Automated Rollback

- Triggered by failed checks or alerts.
- Restores previous stable version.
- Reduces downtime and service impact.



8. Database Compatibility

- Check schema and version compatibility.
- Use migrations and backward compatibility.
- Prevent data loss and service failures.



9. Choosing the Right Strategy

- Assess risk and business impact.
- Use observability and monitoring.
- Prioritize fast recovery and rollback.

Strategy	Best For	Risk Level	Recovery
Rolling	Stateless apps	Low	Fast
Blue-Green	Critical services	Low	Fast
Canary	High-risk changes	Medium	Medium
Progressive	Complex systems	Medium	Medium
Feature Flags	Features/A-B test	Low	Fast

10. Key Takeaways for Production

- ✅ Use the right strategy for the right risk and service.
- ✅ Monitor health signals and set up alerts.
- ✅ Automate rollbacks and keep a stable version ready.
- ✅ Ensure database compatibility before rollout.
- ✅ Apply feature flags for flexibility and control.
- ✅ Use observability to make data-driven decisions.
- ✅ Test, rehearse and document your release process.
- ✅ Focus on fast recovery and minimal user impact.

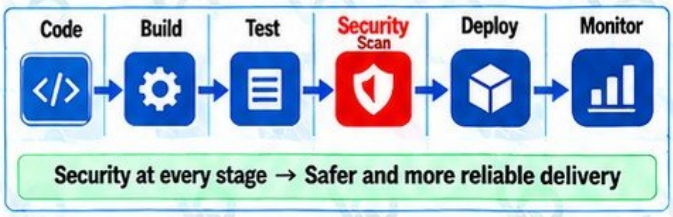
VERIQTA

DevSecOps, Identity and Zero-Trust Infrastructure

IAM, RBAC, workload identity, secrets management, and least privilege.
Network segmentation, TLS, certificate rotation, and admission policies.
Design security controls without creating unnecessary operational fragility.

1. DevSecOps Overview

- Integrate security into every stage of the delivery pipeline.
- Shift left with automated security checks.
- Balance security, speed and operational reliability.



2. Identity and Access Management (IAM)

- Use IAM, RBAC and workload identity for fine-grained access.
- Apply least privilege and avoid long-lived credentials.
- Use roles and service accounts for applications and automation.

Human Users (IAM)
- MFA
- Least privilege
- Role-based access
- Audit logging

Kubernetes RBAC
- Roles / ClusterRoles
- RoleBindings
- Namespace isolation
- Least privilege

Workload Identity
- Service accounts
- OIDC / IRSA
- No long-lived keys
- Scoped permissions

3. Secrets Management

- Store secrets securely using a secrets manager.
- Use short-lived credentials and automatic rotation.
- Avoid hardcoding secrets in code, images or configs.

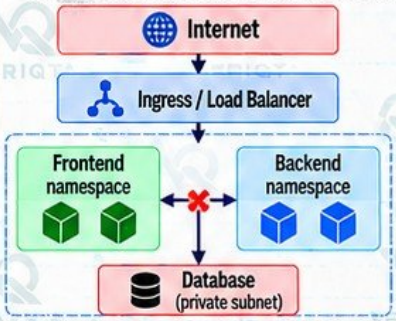
Secrets Options
- Kubernetes Secrets
- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager

Example: Using a Kubernetes Secret

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  username: veriqta
  password: "SuperSecure123"
```

4. Network Segmentation

- Isolate workloads using namespaces, network policies and VPC design.
- Control east-west and north-south traffic.
- Reduce blast radius and lateral movement.



5. TLS and Certificate Management

- Encrypt traffic in transit with TLS.
- Use cert-manager for automated certificate handling.
- Rotate certificates regularly.

Example: cert-manager Certificate

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: app-tls
dnsNames:
  - app.veriqta.com
issuerRef:
  name: letsencrypt-prod
  kind: ClusterIssuer
```

6. Admission Policies and Security Controls

- Use admission controllers to enforce security rules.
- Validate images, resources and configurations.
- Prevent non-compliant or risky deployments.

Common Policies
- ✓ Image allow/deny lists
- ✓ Require resource limits
- ✓ Block privileged containers
- ✓ Enforce non-root user
- ✓ Validate security contexts
- ✓ Use OPA / Gatekeeper / Kyverno

Example: Prevent privileged containers

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-privileged
spec:
  validationFailureAction: Enforce
  rules:
  - name: check-privileged
    match:
      resources:
        kinds: ["Pod"]
    validate:
      message: "Privileged containers
        are not allowed."
      pattern:
        spec:
          containers:
          - (securityContext):
              privileged: false
```

7. Zero-Trust Principles

- Never trust, always verify.
- Use identity, network and device controls.
- Apply continuous verification and monitoring.



8. Best Practices

- ✓ Use least privilege for all identities.
- ✓ Use workload identity instead of static keys.
- ✓ Store secrets in a dedicated secrets manager.
- ✓ Encrypt all traffic with TLS.
- ✓ Implement network segmentation and network policies.
- ✓ Use admission policies to block risky deployments.
- ✓ Rotate certificates and secrets regularly.
- ✓ Monitor, log and alert on security events.
- ✓ Automate security checks in CI/CD pipelines.
- ✓ Balance security with operational reliability and avoid unnecessary downtime.

Key Tools

aws · Azure · Google Cloud · Vault · cert-manager · Kyverno · OPA

9. Trade-offs and Considerations

Decision	Benefit	Risk / Trade-off	When to Use
Tight network segmentation	Reduced blast radius	Higher operational complexity	Sensitive or multi-tenant environments
Strict admission policies	Blocks risky workloads	May block legitimate use cases	Production clusters
Short-lived credentials	Lower credential risk	Requires automation	All environments
Zero-trust model	Stronger overall security	More setup and monitoring	Enterprise and regulated environments

10. Key Takeaways

- ✓ Use identity, network and policy controls together.
- ✓ Protect secrets and rotate credentials.
- ✓ Encrypt and segment your network.
- ✓ Automate security in the delivery pipeline.
- ✓ Apply zero-trust principles without creating operational fragility.
- ✓ Continuously monitor, audit and improve.

VERIQTA

Production Observability and Distributed Tracing

Metrics, logs, traces, correlation IDs, and OpenTelemetry.
Latency percentiles, saturation, high-cardinality metrics, and telemetry costs.
Diagnose failures across multiple services using correlated evidence.

1. Observability Pillars

- Use metrics, logs and traces together.
- Understand how they complement each other.
- Correlate events using a trace ID or correlation ID.

Metrics (What)
- Numbers
- Trends
- Alerting
- e.g. CPU, latency, error rate

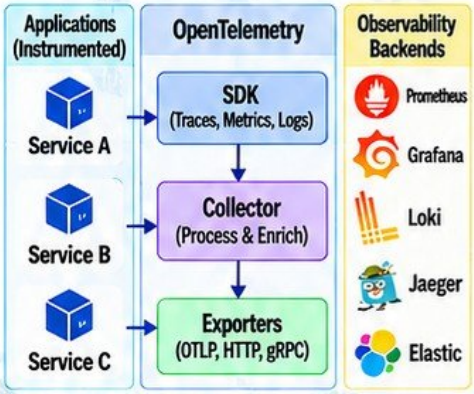
Logs (Why)
- Detailed events
- Debugging
- Error messages
- e.g. application logs, system logs

Traces (How)
- Request flow
- Service calls
- Latency per span
- End-to-end view

Metrics + Logs + Traces → Full visibility

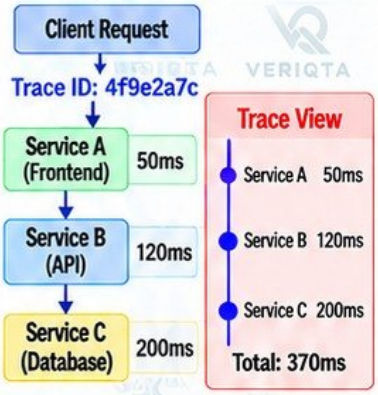
2. OpenTelemetry Architecture

- Instrument your applications with OpenTelemetry.
- Collect, process and export telemetry data.
- Send to your observability backend.



3. Distributed Tracing Flow

- Each request gets a unique trace ID.
- Traces show the path across services.
- Identify slow services and failures.



4. Key Metrics and Latency Percentiles

- Monitor request rate, error rate, latency and saturation.
- Use percentiles (P50, P95, P99) to understand user experience.
- Track resource saturation (CPU, memory, disk, network).

Important Metrics
- Request rate (RPS)
- Error rate (4xx, 5xx)
- Latency percentiles (P50, P95, P99)
- CPU / Memory usage
- Saturation (queue length, connection pool)

Latency Percentiles Example

Percentile	Latency	Meaning
P50	100ms	50% of requests are faster than this
P95	400ms	95% of requests are faster than this
P99	1.2s	99% of requests are faster than this

5. High-Cardinality Metrics

- High-cardinality metrics can cause high cost.
- Avoid using unbounded labels (e.g. user_id).
- Use controlled and meaningful labels.

High Cardinality (Avoid)
- ✗ user_id=12345
- ✗ session_id=abcd
- ✗ request_id=xyz
- ✗ pod_name=xxx
- ✗ unbounded paths

Good Cardinality (Use)
- ✓ service=api
- ✓ environment=prod
- ✓ status_code=500
- ✓ region=us-east-1
- ✓ endpoint=/login

6. Correlation IDs and Logs

- Use a trace ID or correlation ID in all services.
- Include it in logs, metrics and traces.
- Helps you follow a request across services.

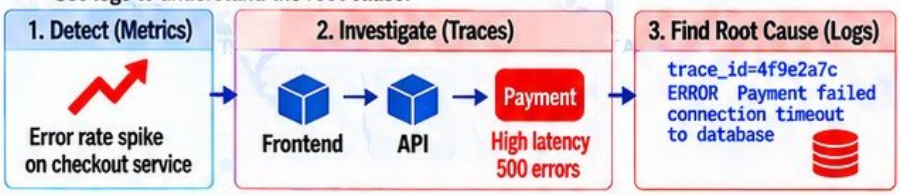
Example Log with Correlation ID

```
2026-10-22T10:24:31Z  INFO
trace_id=4f9e2a7c  span_id=8c1d2e3f
service=payment-api
msg="Payment processed successfully"
user_id=12345  amount=100.00
```

Where to Include the ID
- ✓ Application logs
- ✓ HTTP headers (e.g. traceparent)
- ✓ Metrics labels (controlled)
- ✓ Error reports and alerts

7. Diagnosing Failures with Correlated Evidence

- Use metrics to detect the problem.
- Use traces to find the slow or failing service.
- Use logs to understand the root cause.



Metrics → Traces → Logs → Root Cause → Fix → Prevent Recurrence

8. Observability Stack Example

Visualization	Grafana — Dashboards & Exploration
Metrics	Prometheus — Metrics Collection
Logs	Loki — Log Aggregation
Traces	Jaeger — Distributed Tracing
OpenTelemetry	OpenTelemetry — Instrumentation & Collection

9. Best Practices

- ✓ Instrument all services with OpenTelemetry.
- ✓ Use consistent trace IDs across services.
- ✓ Monitor key business and technical metrics.
- ✓ Set meaningful alerts (not alert fatigue).
- ✓ Control metric cardinality to reduce cost.
- ✓ Use dashboards for latency, errors and saturation.
- ✓ Combine metrics, logs and traces for root cause analysis.
- ✓ Regularly review and tune telemetry costs.

10. Common Tools

Prometheus | Grafana | Loki | Jaeger | OpenTelemetry | Elastic

VERIQTA

SLO Engineering, Error Budgets and Alert Design

User-centered SLIs, SLO targets, and error-budget consumption.
Multi-window burn-rate alerts, alert fatigue, and operational escalation.
Make reliability decisions using customer impact rather than dashboard appearance.

1. Key Concepts

- SLI: A measurable indicator of service performance.
- SLO: Target level for the SLI (e.g. 99.9% availability).
- Error Budget: Allowed amount of failure.
- SLA: External commitment (often includes penalties).

SLI (Measure)
- Availability
- Latency
- Error rate
- Throughput

SLO (Target)
- e.g. 99.9%
- User-centered
- Based on real usage

Error Budget (Allowed Failures)
- 1% for 99% SLO
- 0.1% for 99.9%
- Guides risk decisions

SLA (Contract)
- External target
- Legal/financial
- Often higher than SLO

2. SLI and SLO Examples (User-Centered)

Service	SLI (What to measure)	SLO Target	Why it matters
Web Application	Request success rate (2xx responses)	99.9%	Users can access the app
API Service	Latency (P95)	≤ 300ms	Fast user experience
Payment Service	Successful payments	99.99%	Business critical
Background Jobs	Job completion rate	99.9%	Data processed correctly
Database	Read latency (P95)	≤ 100ms	Application performance
Email Service	Delivery success rate	99.9%	Users receive emails

3. Error Budget and SLO Calculation

- Error Budget = (1 – SLO) × Total Requests
- If SLO is 99.9%, error budget is 0.1% (43 minutes/month).
- Track consumption over time (day, week, month).

Example: 99.9% SLO (30 days)

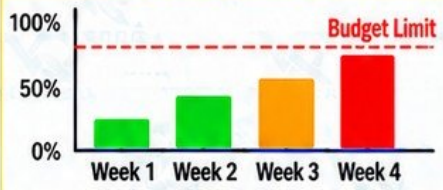
Total time: 30 days = 43,200 minutes
`Allowed downtime (0.1%):`
`43,200 × 0.001 = 43.2 minutes`

- If downtime is 20 minutes → within budget ✓
- If downtime is 60 minutes → budget exceeded ✗

4. Error Budget Consumption

- Track how much of the error budget has been used.
- Use multiple time windows (e.g. 1h, 6h, 1d) for faster detection.
- Make data-driven decisions (slow down releases, fix issues, or continue).

Error Budget Usage Chart (Example)



5. Multi-Window Burn-Rate Alerts

- Use multiple time windows to detect real issues.
- Common pattern: 5m/1h and 1h/6h.
- Detects both fast and slow burns.

Alert Type	Time Window	Burn Rate	Purpose
Page (critical)	5 minutes and 1 hour	≥ 14×	Severe issue, page immediately
Page (major)	1 hour and 6 hours	≥ 6×	Ongoing issue, page on-call
Ticket (warning)	6 hours and 3 days	≥ 3×	Early warning, create ticket
Info (visibility)	1 day and 7 days	≥ 1×	Trend analysis, no page

6. Alert Design Best Practices

- ✓ Alert on user impact (SLIs), not just infrastructure metrics.
- ✓ Use multiple time windows and thresholds.
- ✓ Avoid alert fatigue (reduce noise).
- ✓ Include clear runbooks and escalation paths.
- ✓ Route alerts to the right team.
- ✓ Use meaningful alert names and descriptions.
- ✓ Test alerts regularly.
- ✓ Silence alerts during planned maintenance.

7. Alert Fatigue vs Good Alerts

Alert Fatigue (Avoid)
- ✗ Too many low-value alerts
- ✗ Alerts on single instances
- ✗ Noisy metric thresholds
- ✗ No context or runbook
- ✗ Paging for non-critical issues
- ✗ Teams start ignoring alerts

Good Alerts (Recommended)
- ✓ Alert on SLO burn rate
- ✓ User-impact focused
- ✓ Meaningful thresholds
- ✓ Include runbook and context
- ✓ Correct escalation path
- ✓ Keep alerts actionable

8. Operational Escalation

Alert Triggered (SLO breach)
↓
On-Call Responds (Investigate and mitigate)
↓
Escalate if needed (Secondary/Engineering Manager)
↓
Resolve and Review (Post-incident analysis)

9. Reliability Decisions Based on Customer Impact

Scenario	Customer Impact	Decision	Reason
Error budget healthy	Low	Continue normal releases	Service is stable
Error budget at risk	Moderate	Slow down or add safeguards	Protect reliability
Error budget exhausted	High	Stop releases and focus on fixes	Prevent further impact
Critical user-facing issue	High	Page immediately	User impact is severe

10. Key Takeaways

- ✓ Define user-centered SLIs and realistic SLO targets.
- ✓ Track and monitor error budget consumption.
- ✓ Use multi-window burn-rate alerts to detect issues early.
- ✓ Reduce alert fatigue and keep alerts actionable.
- ✓ Follow clear escalation paths and runbooks.
- ✓ Make reliability decisions based on actual customer impact.
- ✓ Continuously review and improve your SLOs and alerting.

VERIQTA

Incident Management and Root-Cause Investigation

Incident severity, blast radius, mitigation, and command structure.
Hypothesis-driven debugging, timelines, evidence collection, and postmortems.
Restore service safely while preserving information needed for investigation.

1. Incident Lifecycle

- Detect and validate the incident.
- Assess severity and blast radius.
- Mitigate and restore service.
- Investigate root cause.
- Document and run a postmortem.



2. Incident Severity Levels

Severity	Impact	Examples	Response Time
SEV 1	Major outage	• Service down for many users • Data loss risk	Immediate (0–15 min)
SEV 2	Severe degradation	• High error rates • Key feature unavailable	< 1 hour
SEV 3	Limited impact	• Non-critical feature issue • Workaround exists	< 4 hours
SEV 4	Minor issue	• Small impact • Cosmetic issue	Next business day

3. Incident Command Structure

- Clear roles and responsibilities.
- Single source of truth for updates.
- Regular communication cadence.
- Decision making based on data.

Incident Commander
Leads response and decisions

Technical Lead
Drives investigation and mitigation

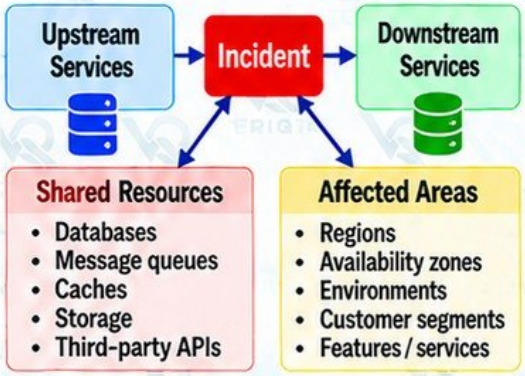
Communications
Updates stakeholders and users

Scribe
Records timeline, decisions and actions

Subject Matter Experts
Provide technical support

4. Blast Radius Analysis

- Identify what is affected and what is not.
- Understand dependencies and shared resources.
- Check downstream and upstream impact.
- Prevent further impact before making changes.



5. Hypothesis-Driven Debugging

- Start with the most likely hypotheses.
- Use evidence to confirm or reject.
- Follow the data, not assumptions.
- Test one change at a time.



6. Evidence Collection

- Collect logs, metrics, traces, and events.
- Preserve system state before changes.
- Capture configurations and recent deployments.
- Keep a timeline of all actions.

Evidence to Collect

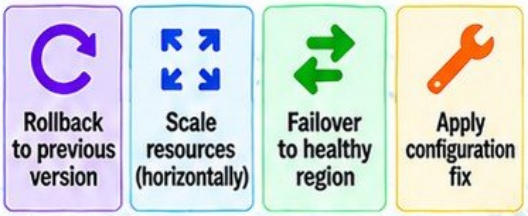
- ✓ Application logs
- ✓ Infra and system logs
- ✓ Metrics and dashboards
- ✓ Distributed traces
- ✓ Recent deployments / changes
- ✓ Configuration files
- ✓ Network and firewall logs
- ✓ Screenshots and timelines
- ✓ Customer reports and impact data

Preserve Before Making Changes

- Take snapshots
- Export logs
- Capture state
- Document commands
- Avoid losing evidence

7. Mitigation and Recovery

- Apply the safest fix first.
- Consider rollback, scaling or failover.
- Verify service health after the fix.
- Monitor for recurrence.



Verify → Monitor → Confirm Stability

8. Postmortem and Follow-up

- Write a blameless postmortem.
- Identify root cause and contributing factors.
- Define action items with owners and due dates.
- Prevent recurrence with long-term fixes.

Postmortem Sections

1. What happened? (summary)
2. Impact (users, services, duration)
3. Timeline (key events with times)
4. Root cause (technical and process)
5. Contributing factors
6. What went well?
7. What can be improved?
8. Action items (owner and due date)

9. Common Root Causes

- Changes and deployments.
- Configuration errors.
- Resource exhaustion.
- Dependency failures.
- Code bugs and memory leaks.
- Network and DNS issues.
- Third-party service outages.
- Insufficient monitoring or alerting.
- Human error and process gaps.

10. Key Takeaways

- ✓ Respond quickly and stay calm.
- ✓ Use data to make decisions.
- ✓ Communicate clearly and frequently.
- ✓ Preserve evidence for investigation.
- ✓ Learn from every incident and improve.
- ✓ Focus on customer impact, not dashboards.

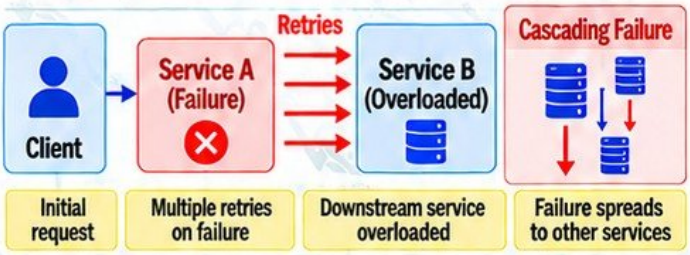
VERIQTA

Distributed Systems Failure Patterns and Resilience

Cascading retries, retry storms, circuit breakers, and backpressure. Queues, consistency trade-offs, idempotency, and partial failures. Design bounded failure behavior and graceful degradation.

1. Common Failure Patterns

- Retries can amplify failures and overload downstream services.
- Understand how failures propagate across services.
- Design systems to limit blast radius and recover gracefully.



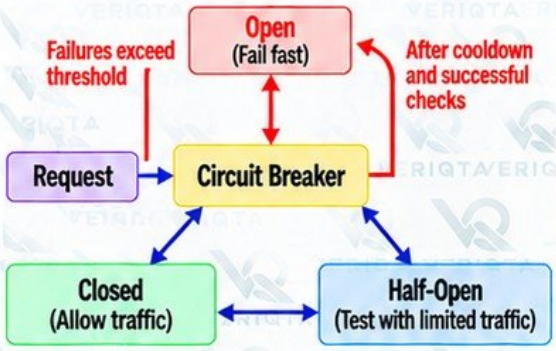
2. Retry Storms and Backpressure

- Uncontrolled retries can create a retry storm.
- Use exponential backoff, jitter, and retry limits.
- Apply backpressure to shed load and protect critical services.



3. Circuit Breaker Patterns

- Stop calls to failing services to prevent cascading failures.
- Allow recovery after a cooldown period.
- Use thresholds based on failure rate and latency.



4. Queues and Asynchronous Processing

- Use queues to decouple services.
- Handle spikes and slow downstream systems.
- Design for at-least-once delivery and idempotency.



Benefits

- ✓ Smooths traffic spikes
- ✓ Decouples services
- ✓ Improves resilience
- ✓ Allows independent scaling

Considerations

- ✓ At-least-once delivery
- ✓ Handle duplicate messages
- ✓ Design idempotent consumers
- ✓ Monitor queue depth and lag

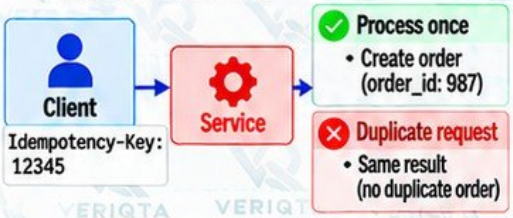
5. Consistency Trade-offs

- Choose the right consistency model for your use case.
- Understand the trade-offs between consistency, availability and partition tolerance (CAP).
- Use eventual consistency where appropriate.

Strong Consistency	• Always up-to-date • Higher latency, lower availablliity
Eventual Consistency	• Updates propagate over time • Higher availability, lower latency
Read-After-Write	• Read your own writes • Common in distributed systems
Write Conflicts	• Use versioning or optimistic • locking to handle conflicts

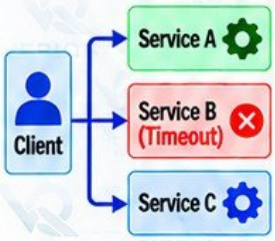
6. Idempotency

- Operations should be safe to retry multiple times.
- Use idempotency keys to prevent duplicate processing.
- Important for payments, messaging and distributed workflows.



7. Partial Failures and Timeouts

- In distributed systems, some services will fail while others succeed.
- Use timeouts, fallbacks, and retries with limits.
- Design for partial failures, not just total failure.

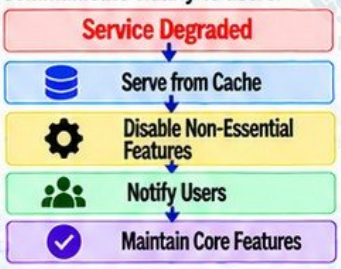


Strategies

- ✓ Set timeouts
- ✓ Use fallbacks
- ✓ Return partial results
- ✓ Retry with limits
- ✓ Isolate failures
- ✓ Provide meaningful error messages

8. Graceful Degradation

- Maintain core functionality during failures.
- Use feature flags, reduced functionality, or cached responses.
- Communicate clearly to users.



9. Real-World Examples

Failure Pattern	Example	Impact	Mitigation
Cascading retries	Service A retries Service B	Overload and full outage	Retry limits, backoff, circuit breaker
Retry storm	Many clients retry at once	Downstream collapse	Exponential backoff, jitter, rate limits
Partial failure	One service times out	Degraded functionality	Timeouts, fallbacks, partial responses
Queue buildup	Consumer is slow	Growing latency, memory issues	Scale consumers, backpressure
Data inconsistency	Replica lag in database	Stale reads	Use eventual consistency, read-after-write
Duplicate processing	Retry without idempotency	Multiple charges/orders	Idempotency keys, deduplication

10. Key Takeaways

- ✓ Expect failures in distributed systems.
- ✓ Design for bounded failure behavior.
- ✓ Use retries, backoff, and circuit breakers.
- ✓ Apply backpressure to protect critical services.
- ✓ Choose the right consistency model.
- ✓ Use idempotency and handle partial failures.
- ✓ Design graceful degradation and clear communication.
- ✓ Monitor, test, and regularly validate resilience.

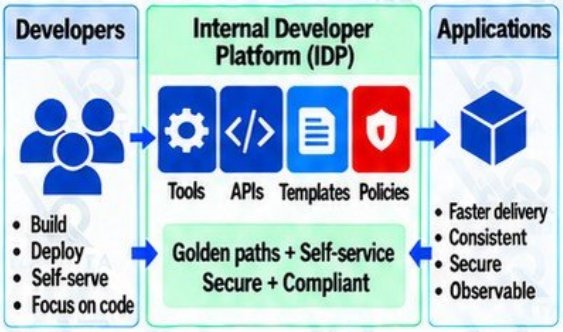
VERIQTA

Platform Engineering and Internal Developer Platforms

Golden paths, self-service infrastructure, platform APIs, and ownership boundaries. Developer experience, policy-as-code, service templates, and platform observability. Build platforms that reduce cognitive load without hiding critical operational behavior.

1. What is a Platform Engineering?

- A set of reusable tools, services and workflows.
- Provides self-service infrastructure for developers.
- Enforces security, compliance and best practices.
- Reduces cognitive load and accelerates delivery.



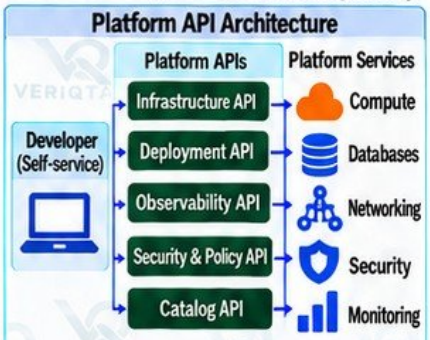
2. Golden Paths and Self-Service

- Provide easy, supported ways to do the right thing.
- Abstract complexity without hiding important details.
- Offer reusable templates and automated workflows.



3. Platform APIs and Services

- Expose internal services through APIs.
- Standardize access and authentication.
- Provide clear documentation and examples.
- Use versioning and backward compatibility.

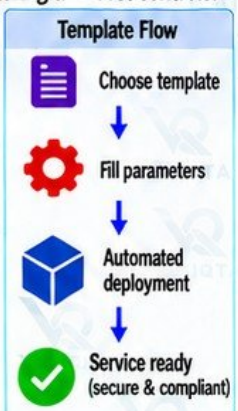


4. Service Templates and Automation

- Use templates to create consistent, compliant services.
- Automate provisioning, configuration and CI/CD.
- Include built-in security, monitoring and cost controls.

Service Template Example

```
apiVersion: platform.veriqta.io/v1
kind: ServiceTemplate
metadata:
    name: web-service
spec:
  parameters:
    - name: service_name
    - name: environment
  defaults:
    replicas: 2
    monitoring: true
    logging: true
    network: private
    policy: baseline
```



5. Policy-as-Code

- Define guardrails using code.
- Enforce security, compliance and cost rules.
- Block non-compliant deployments automatically.
- Give clear feedback to developers.



Example Policy Rules

- ✓ Use approved regions
- ✓ Enforce encryption
- ✓ Require resource tags
- ✓ Block public storage
- ✓ Limit instance types
- ✓ Require monitoring
- ✓ Enforce network policies
- ✓ Set cost limits

6. Developer Experience (DX)

- Make it easy to find, use and understand platform services.
- Provide clear documentation, examples and support.
- Reduce context switching and tool sprawl.
- Get feedback from developers and continuously improve.



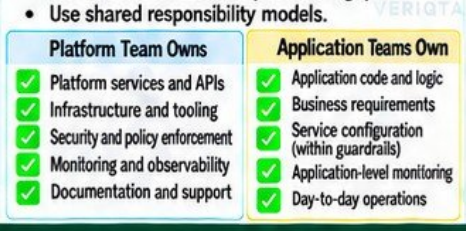
7. Platform Observability

- Monitor platform services and user adoption.
- Track performance, reliability and cost.
- Provide visibility for both platform team and developers.



8. Ownership Boundaries

- Define clear ownership between platform team and application teams.
- Prevent confusion and operational gaps.
- Use shared responsibility models.



9. Benefits and Trade-offs



10. Key Takeaways

- ✓ Provide golden paths and self-service infrastructure.
- ✓ Use platform APIs, templates and automation.
- ✓ Enforce security and compliance with policy-as-code.
- ✓ Focus on developer experience and platform observability.
- ✓ Define clear ownership boundaries.
- ✓ Reduce cognitive load without hiding critical operational behavior.
- ✓ Continuously improve based on developer feedback.
- ✓ Build a platform that enables fast, safe and reliable delivery.

VERIQTA

Senior DevOps Architecture Review and Production Readiness

Evaluate reliability, security, scalability, performance, cost, and operational ownership. Review architecture decisions, disaster recovery, capacity, deployment safety, and observability. Final production scenario: assess a multi-service platform, identify architectural risks, prioritize improvements, and defend engineering trade-offs.

1. Architecture Review Framework

- Evaluate the full stack: people, process, and technology.
- Use a structured approach and evidence, not opinion.
- Check reliability, security, scalability, performance, cost, and operational ownership.
- Identify risks, gaps, and improvement opportunities.

Reliability
- Availability
- Resilience
- Failure recovery

Security
- IAM and access
- Data protection
- Threat mitigation

Scalability
- Horizontal scaling
- Auto scaling
- Capacity planning

Performance
- Latency
- Throughput
- Resource usage

Cost
- Right sizing
- Cost controls
- Efficiency

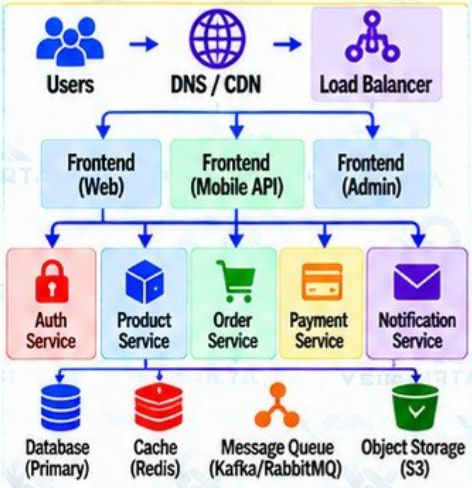
Operations
- Ownership
- On-call model
- Runbooks

Deployment
- Safety
- Rollback strategy
- Change management

Observability
- Metrics, logs, traces
- Alerting
- Dashboards

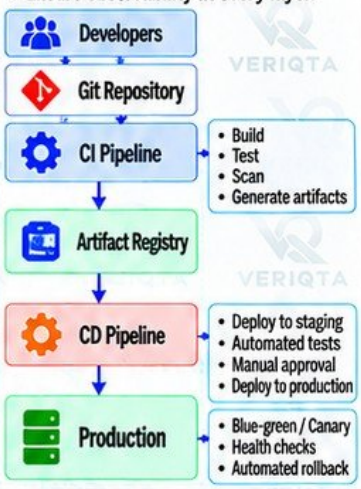
2. Example Multi-Service Platform

- E-commerce style application with multiple services.
- Deployed across multiple environments.
- Uses container orchestration and managed cloud services.
- Includes CI/CD, monitoring, and disaster recovery.



3. Deployment Architecture

- Use environments: dev, staging, prod.
- Automate deployments with CI/CD.
- Use immutable infrastructure.
- Implement safe rollout strategies.
- Ensure observability at every layer.



4. Disaster Recovery and Business Continuity

- Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO).
- Use multi-az or multi-region deployments.
- Test backups and restore procedures regularly.
- Document and automate disaster recovery runbooks.

Backup Strategy
- Automated backups
- Regular testing
- Encrypted storage
- Retention policy

Multi-Region Setup

Primary Region ↔ Secondary Region
- Automatic failover
- DNS-based routing
- Data replication

DR Runbook
- Step-by-step procedures
- Roles and responsibilities
- Communication plan
- Regular DR drills

5. Capacity and Scaling Review

- Analyse current and future demand.
- Identify bottlenecks and single points of failure.
- Use load testing and real production metrics.
- Plan for horizontal and vertical scaling.

Scaling Strategies

Horizontal Scaling
- Add more instances
- Auto scaling

Vertical Scaling
- Increase CPU/RAM
- Useful for stateful services

Load Balancing
- Distribute traffic
- Health checks

Caching
- Reduce load
- Improve performance

6. Security Review

- Apply least privilege access.
- Use IAM, RBAC and workload identity.
- Protect secrets and sensitive data.
- Enforce network segmentation.
- Scan for vulnerabilities and misconfigurations.

Key Security Areas

Identity and Access	IAM, RBAC, SSO Least privilege
Secret Management	Encrypted storage Automatic rotation
Network Security	VPC, security groups Private endpoints
Compliance	Security scanning Policy as code
Supply Chain Security	Signed artifacts Dependency scanning (SBOM)

7. Production Readiness Checklist

- ✅ All services have monitoring and alerting.
- ✅ Runbooks and escalation paths are documented.
- ✅ Security controls are implemented and tested.
- ✅ Disaster recovery plan is tested.
- ✅ Capacity and scaling are validated.
- ✅ Deployment strategy is safe and automated.
- ✅ Costs are reviewed and optimized.
- ✅ Operational ownership is clear.
- ✅ Final architecture review completed.

8. Risk Assessment and Prioritization

Risk	Impact	Likelihood	Priority	Action
Single point of failure	High	Medium	High	Add redundancy
Data loss	High	Low	High	Improve backup/DR
Security breach	High	Medium	High	Strengthen IAM
Performance degradation	Medium	High	High	Optimize and scale
High cloud costs	Medium	High	Medium	Right size resources
Deployment failure	High	Medium	High	Improve CI/CD safety
Lack of observability	High	Medium	High	Add metrics, logs, traces

9. Final Scenario and Tasks

- You are the lead engineer reviewing a multi-service e-commerce platform.
- Identify architectural risks.
- Prioritize the top improvements.
- Explain and defend your trade-offs.
- Present your recommendations to stakeholders.

What to Deliver
- Architecture review summary
- Top risks and recommended fixes
- Justification for each decision
- Updated architecture diagram
- Production readiness checklist

10. Key Takeaways

- ✅ Review architecture with a holistic view.
- ✅ Use real metrics and evidence.
- ✅ Design for reliability, security and scalability.
- ✅ Plan and test disaster recovery.
- ✅ Automate safe and observable deployments.
- ✅ Identify and prioritize risks.
- ✅ Make informed trade-offs.
- ✅ Make operational tradeoffs.
- ✅ Ensure operational readiness.
- ✅ Reduce cognitive load for teams.
- ✅ Build resilient, secure and cost-effective platforms.